

西南大学

数学建模竞赛论文

论文题目：基于时序外推与稳健多目标优化的甘蔗产量决策研究

成员：赵武岳、祝铭涵、马宁

选题：B 题

姓名	学院	年级	专业	学号	联系电话
赵武岳	工程技术学院	2025	电气工程及其自动化	222024323012022	19823363272
祝铭涵	计算机与科学技术学院	2024	自动化	222024321132036	19888703694
马宁	计算机与科学技术学院	2024	自动化	222024321132031	18663152772

日期：2026 年 7 月 28 日

承 诺 书

我们仔细阅读了《全国大学生数学建模竞赛章程》和《西南大学数学建模模拟赛作品提交要求》(以下简称“竞赛章程和参赛规则”)。

我们完全清楚,在竞赛开始后参赛队员不能以任何方式,包括电话、电子邮件、“贴吧”、QQ 群、微信群,线下交流等,与队外的任何人(包括指导教师)交流、讨论与赛题有关的问题;无论主动参与讨论还是被动接收讨论信息都是严重违反竞赛纪律的行为。

我们以西南大学大学生名誉和诚信郑重承诺,严格遵守竞赛章程和参赛规则,以保证竞赛的公正、公平性。如有违反竞赛章程和参赛规则的行为,我们将受到严肃处理。

我们以西南大学大学生名誉和诚信郑重承诺,所给出论文中应为原创内容,如有查重过高并被检测出的行为,我们将受到严肃处理。

我们参赛选择的题号(从 A/B/C 中选择一项填写): B

参赛队员(电子签名): 1. 赵武岳

2. 祝铭涵

3. 马宁

指导教师(电子签名,没有则不填): 赵仲勇

(指导教师签名意味着对参赛队的行为和论文的真实性负责)

日期: 2026 年 7 月 28 日

基于时序外推与稳健多目标优化的甘蔗产量决策研究

摘要：针对 3000 个甘蔗地块的气候、土壤、水肥管理、品种和产量记录，本文建立“数据审计—产量响应—约束决策—独立复算”的统一框架，依次研究因素关系、水肥优化、气候鲁棒决策、产量—环境权衡及品种—季节适应性。首先将 2018—2022 年作为训练期、2023 年作为模型选择期、2024 年作为封存测试期，并把插补、编码和标准化限定在训练折内；日期矛盾、土壤组成矛盾及生长中后期变量不进入前瞻决策模型，从而控制数据泄漏。

问题一比较均值基线、Elastic Net、EBM 与 CatBoost。依据“复杂模型的验证集 RMSE 至少改善 5% 才替代简约模型”的预设规则，最终保留 Elastic Net；其 2023 年 RMSE 为 79.720、 $R^2 = 0.391$ ，锁定后在 2024 年取得 RMSE 为 81.094、 $R^2 = 0.385$ 。置换重要性显示氮含量、土壤条件、钾含量和土壤水分最重要，但本文仅将其解释为稳定预测关联。

问题二以 Gower 距离选取真实中心样本作为典型地块，在观测支持域与 2500 元/公顷预算内进行密集网格和局部复核。模型给出的候选解为灌溉量 1917.530 m³/英亩、施肥量 62.886 kg/英亩，预测产量 250.262 公担/英亩，成本 1413.85 元/公顷；因水肥效应方向和模型结构存在不确定性，该结果界定为边界候选而非无条件最优。问题三以稳健标准化、PCA 和 K-medoids 构造三个真实气候情景，max-min 枚举得到品种 Co98014 与灌溉量 1917.530 m³/英亩，三情景产量分别为 253.376、266.704 和 291.580 公担/英亩；响应模型重采样揭示品种选择不稳定，故同时给出候选比例。

问题四构造水肥归一化平方环境压力，用增广 ϵ 约束思想提取 351 个 Pareto 点，并以到理想点的归一化距离选择折中解：灌溉 1364.326 m³/英亩、施肥 62.886 kg/英亩、产量 249.260 公担/英亩、环境压力 0.1732；NSGA-II 复核的前沿中位距离为 0.0012。问题五用混合效应模型校正气候、土壤与管理差异，经冗余筛选后以 CRITIC-TOPSIS 综合调整均值、稳定性和下尾产量。前三个并列优先候选为 CoJ64-Kharif、CoJ64-Rabi 和 Co0238-Rabi，其 Top-3 概率均低于 70%，因此不作确定性名次承诺。最终 27 项独立核验无失败项，并明确单位解释、观察数据与模型外推的适用边界。

关键词：甘蔗产量；Elastic Net；鲁棒优化；Pareto 前沿；混合效应模型；CRITIC-TOPSIS

一、 问题重述与分析

1.1、 研究对象与系统边界

题目给出某甘蔗主产区 3000 个“地块—生长季”样本，共 81 个字段，覆盖 2018—2024 年。数据包含气候、土壤、水肥管理、种植措施、病虫害、生长状态、品种与最终产量。本文把单个地块的一个生长季作为基本研究单元，输出变量为最终产量 Y （公担/英亩）。需要特别说明：数据中每行只有一个月份标签和一组气候记录，并不是“同一地块连续 12 个月”的完整面板。因此本文不能构造逐月递推生长模型，而采用记录级联合气候状态；这一边界贯穿全部五问。

为了避免把收获前才能观察到的信息用于种植前决策，将变量按时间顺序分为：种植前固定条件、种植与管理决策、生长季气候、中后期生长状态和最终产量。发芽率、分蘖数、株高、茎径、节间长度、Brix、病虫害和生育阶段只可用于事后辅助解释，不能进入问题二至问题四的前瞻响应模型。

1.2、 五个问题的任务重述

问题 1： 从气候、土壤、管理和品种等信息中识别对最终产量预测最重要的因素，分析其效应方向及可能的农业解释；同时避免把预测关联误写成因果关系。

问题 2： 选取一个真实且具有代表性的地块，在水费与肥料费不超过预算的条件下，确定可调整的灌溉量和施肥量，使模型预测产量最大，并复算产量、成本与约束余量。

问题 3： 基于历史气候联合分布构造三个典型情景，在品种与灌溉量组成的决策集中最大化最差情景产量，并分别报告三种情景产量及鲁棒代价。

问题 4： 构造可由现有数据计算的环境成本代理量，形成“最大化产量—最小化环境压力”的双目标模型，得到 Pareto 前沿，并按照透明规则选取折中点。

问题 5： 提取全部“品种—季节”组合，在校正地块气候、土壤和管理差异后评价平均表现、下尾风险与稳定性，报告前三个组合及第一候选的适宜条件，同时量化排名不确定性。

1.3、 总体分析框架

五问不是五套互不相干的模型。本文先建立统一的前瞻产量响应函数

$$\hat{Y} = f(\mathbf{x}_{\text{soil}}, \mathbf{x}_{\text{climate}}, \mathbf{x}_{\text{manage}}, v), \quad (1.1)$$

再将其嵌入水肥约束、气候 max-min 和环境双目标决策。这样，优化算法只负责提出决策，产量由同一已验证响应模型给出，成本、情景汇总和环境指标则由独立复算器重新计算。

针对问题一，线性正则模型可以稳定处理共线性并给出基线方向，可解释提升模型能够描述非线性，CatBoost 给出复杂模型性能上界。三者必须在同一时间切分下竞争。针对问题二，决策变量只有两个，密集网格能覆盖整个支持域并直接审计，局部搜索只用于精化而不是代替全局检查。针对问题三，气候变量高度相关，应先降维再聚类，并选真实样本作为中心，避免用独立分位数拼接出不可能的气候。针对问题四，加权和可能遗漏非凸前沿，因此用 ε 约束式枚举为主、NSGA-II 为对照。针对问题五，原始均值受地块条件混杂，需先作协变量校正，再进行多指标评价和重采样排名。

整个流程采用 2018—2022 年训练、2023 年选择、2024 年封存测试。所有缺失插补、编码和标准化均在训练折内部拟合。模型输出分为“已验证”“暂定”“有局限支持”三档：数值可复现不等于农业结论确定，边界解、模型分歧和低入选概率均必须在正文中暴露。

二、 数据预处理、模型假设与符号说明

2.1、 原始数据审计

原始 CSV 含 3000 行、81 列，目标产量无缺失且无完全重复行。只有 88 行完全无缺失，每行平均缺失 3.402 个字段，故若完整案例删除将损失 97.07% 样本。日期与土壤组成存在明显逻辑冲突：2647 条可比日期中有 706 条收获早于种植，2675 条可计算土壤比例中有 2161 条不满足砂、粉、黏之和位于 95%–105%。这些问题不适合通过强行修补得到“看似完整”的数据。

据此实施以下清洗规则：原始数据只读；数值缺失用训练折中位数插补并添加缺失

指示，分类缺失设为 **Unknown**；常量列与地块编号不作普通预测因子；原始种植/收获日期及矛盾的砂粉黏比例从主模型中删除；连续变量不按 IQR 规则机械删极值；所有转换封装进交叉验证管道。清洗后保留 56 个决策安全特征，训练、验证、测试样本量分别为 2085、415 和 403，年份缺失记录不参加主时间外验证。

水量字段名写作“升/英亩”，但其数值范围与甘蔗季节总需水量明显不一致。结合量级，本文按“千升/英亩”，即数值上等于 $\text{m}^3/\text{英亩}$ 解释；施肥量按 $\text{kg}/\text{英亩}$ 解释。换算关系为

$$W_{ha} = 2.47105W, \quad F_{ha} = 2.47105F. \quad (2.1)$$

所有真实成本结论均依赖该解释，并另作价格与预算敏感性分析。

2.2、 模型假设

- H1. 同一字段在不同年份采用统一测量口径；缺失可在给定已观测变量和缺失指示后进行条件插补，但不假定完全随机缺失。
- H2. 2018—2024 年的观测机制总体可比；允许时间漂移，因此以年份外推检验而非随机拆分估计泛化能力。
- H3. 水量与施肥量只在训练期 5%–95% 分位支持域内调整；不把响应模型外推到无样本区域。
- H4. 水价基准为 0.20 元/ m^3 ，肥料综合价基准为 3.00 元/kg，预算为 2500 元/公顷；这些外部假设通过多档价格和预算检验。
- H5. 环境压力缺少氮淋失、能耗或碳排放实测值，故只能建立无量纲水肥投入代理量，不能解释为真实排放或货币环境损失。
- H6. 观测数据支持条件预测和比较，不足以识别严格因果效应；“正向/负向”均表示在所建模型和样本支持域内的预测关联。

2.3、 主要符号

表 1 主要符号及含义

符号	含义	单位或范围
$Y, \hat{Y}$	实际产量与模型预测产量	公担/英亩
$\mathbf{x}$	气候、土壤、管理等决策安全协变量	混合量纲
v	甘蔗品种	Co0238、 Co98014、CoJ64
W, W_{ha}	每英亩与每公顷灌溉量	$\text{m}^3/\text{英亩}$, $\text{m}^3/\text{公顷}$
F, F_{ha}	每英亩与每公顷施肥量	$\text{kg}/\text{英亩}$, $\text{kg}/\text{公顷}$
p_w, p_f	水价与肥价	$\text{元}/\text{m}^3$, $\text{元}/\text{kg}$
B	水肥总预算	$\text{元}/\text{公顷}$
s	气候情景编号	$s = 1, 2, 3$
π_s	情景发生概率	$[0, 1]$, 和为 1
$E(W, F)$	无量纲环境压力代理量	$[0, 1]$ 附近
λ_w	环境压力中水投入权重	$[0, 1]$
d_i^+, d_i^-	TOPSIS 方案到正、负理想点的距离	非负

三、 问题一：产量响应模型与关键因素

3.1、 候选模型与无泄漏训练

以 2018—2022 年数据建立训练集，按年份分组交叉验证调参；2023 年只用于模型选择，2024 年在选择规则冻结后仅开启一次。对每个候选模型均使用同一决策安全特征集和相同折划分，防止因数据口径不同造成虚假的性能差。

基线一为训练期均值。基线二采用 Elastic Net，其目标函数为 [1]

$$\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \left[\alpha \|\boldsymbol{\beta}\|_1 + \frac{1 - \alpha}{2} \|\boldsymbol{\beta}\|_2^2 \right] \right\}. \tag{3.1}$$

其中 L1 项完成稀疏筛选，L2 项缓解土壤养分、降雨和湿度之间的共线性。网格搜索得到 $\alpha = 1.0$ 、 l_1 比例 0.95。

主候选为 Explainable Boosting Machine (EBM)，以多个一元形状函数和少量二元交互表示非线性：

$$\hat{Y} = \beta_0 + \sum_j f_j(x_j) + \sum_{(j,k) \in \mathcal{I}} f_{jk}(x_j, x_k). \quad (3.2)$$

高阶候选 CatBoost 通过有序提升处理非线性与分类变量 [2]。为了防止“复杂即优秀”，预先规定：若 2023 年最优复杂模型相对 Elastic Net 的 RMSE 改善不足 5%，保留 Elastic Net。模型选定后再用 2018—2023 年重拟合，预测封存的 2024 年。

3.2、 模型比较与最终选择

表 2 2023 年时间外验证的候选模型比较

模型	RMSE	MAE	R^2	NRMSE	MAPE/%
均值基线	102.629	85.142	-0.009	0.356	43.40
Elastic Net	79.720	67.851	0.391	0.276	31.44
EBM	80.362	68.888	0.381	0.278	31.17
CatBoost	79.577	67.980	0.393	0.276	30.77

CatBoost 相对 Elastic Net 的 RMSE 只改善 0.179%，明显小于 5% 阈值，EBM 反而略差，因此选用 Elastic Net 作为统一响应模型。该选择并非否定非线性算法，而是在精度相当时减少结构不确定性。2018—2023 年重拟合后的 2024 年封存测试结果为 RMSE=81.094、MAE=67.444、 $R^2 = 0.385$ 、NRMSE=0.290、MAPE=31.81%，平均偏差仅 -0.789 公担/英亩。2023 与 2024 性能接近，说明模型没有因单一验证年份获得偶然优势。

若把株高、Brix 等全生长季信息加入 CatBoost，2024 年 R^2 可达 0.904，但这些变量在种植决策时不可得。故该结果仅说明中后期表型含有大量产量信息，不参与后续优化。

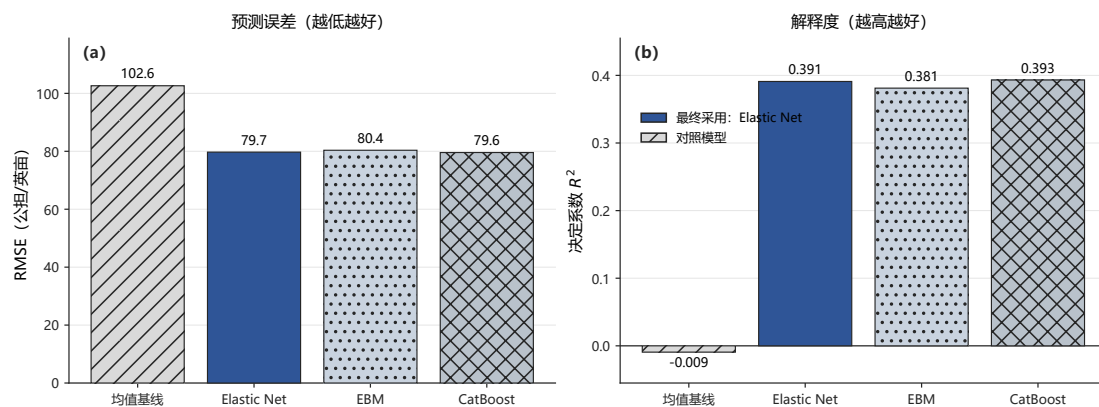


图 1 2023 年时间外验证集上的候选模型性能 (柱顶为指标值, 蓝色为最终采用模型)

3.3、 关键因素与效应方向

对 2023 年样本重复置换单个变量, 计算 RMSE 增量。表 3 列出前七项; 正重复率均为 100%, 说明它们对外年预测的贡献较稳定。

表 3 Elastic Net 置换重要性与方向对比

因素	RMSE 增量均值	标准差	Q90 相对 Q10	预测变化
氮含量	24.088	1.960		+126.766
种植时土壤状态	6.456	0.945	湿润相对干燥	+52.680
钾含量	5.792	0.973		+58.786
土壤含水率	2.579	0.621		+35.740
种植方式	1.364	0.502	沟植相对双行	+24.211
总降雨量	0.783	0.290		+18.344
湿度	0.540	0.193		+11.197

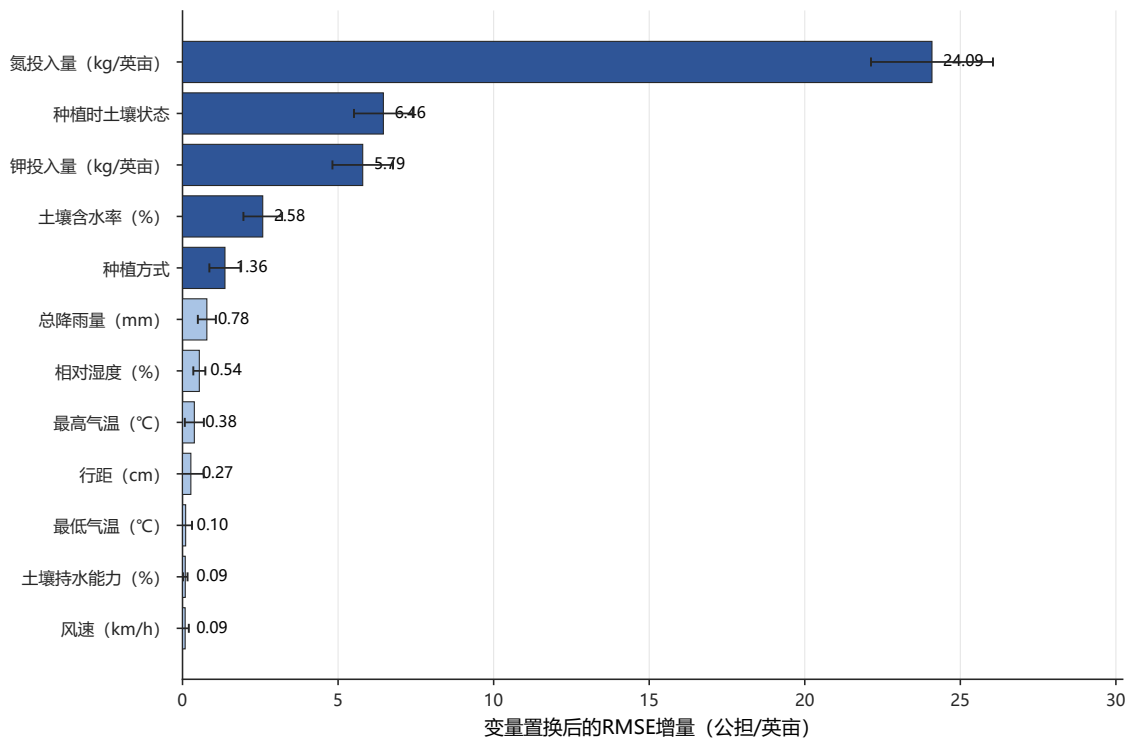


图 2 2023 年时间外置换重要性（误差线为 20 次重复置换的标准差）

氮和钾是蔗茎生物量形成的重要养分，在本样本范围内均呈显著正向预测关联；氮的重要性远高于其他变量，说明不同地块的氮水平是模型区分高低产的主要依据。湿润种植床、较高土壤含水率和较多降雨共同缓解水分胁迫，与甘蔗长生育期需水规律一致[3]。沟植能够改善保墒和水肥集中利用，在该地区样本中相对双行种植表现更高。湿度的正向作用较弱，可能既包含减少蒸散的益处，也混合了病害风险，因此不能作单一机制解释。

值得注意的是，把水量从 10% 分位提高到 90% 分位，Elastic Net 的平均预测只增加 0.274；肥量反而变化 -1.839。这不表示“灌溉无效、施肥有害”，而是说明在控制氮磷钾、土壤水分和其他管理变量后，记录中的总量字段未表现出稳定的独立边际效应。其原因可能是水肥总量单位、农户逆向调整和共线性混杂。故问题二至四必须增加响应模型重采样和边界警告。

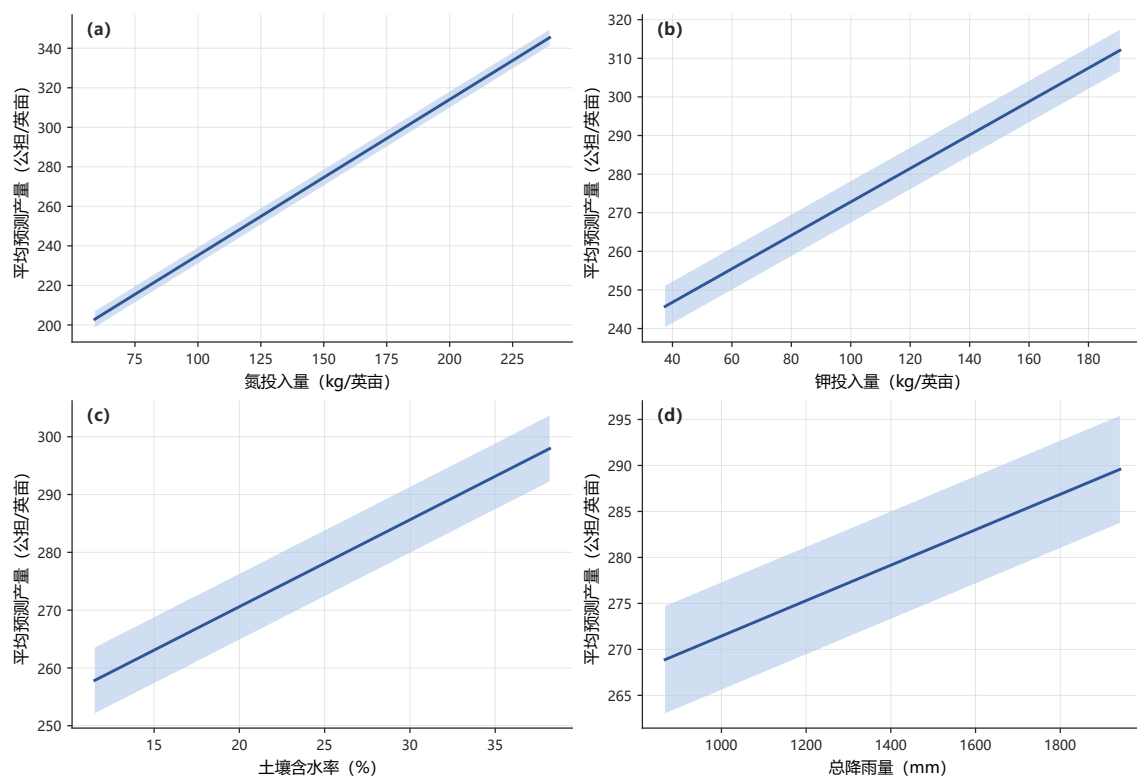


图 3 关键连续变量在观测支持域内的条件效应（阴影为 95% 置信带）

残差诊断中，绝对残差与预测值的 Spearman 相关系数为 0.045，未呈现强烈单调异方差；但单样本误差仍较大， R^2 约 0.39 也表明尚有未观测因素。综上，问题一给出的是经外年验证的预测关系，而不是田间随机试验意义上的因果剂量反应。

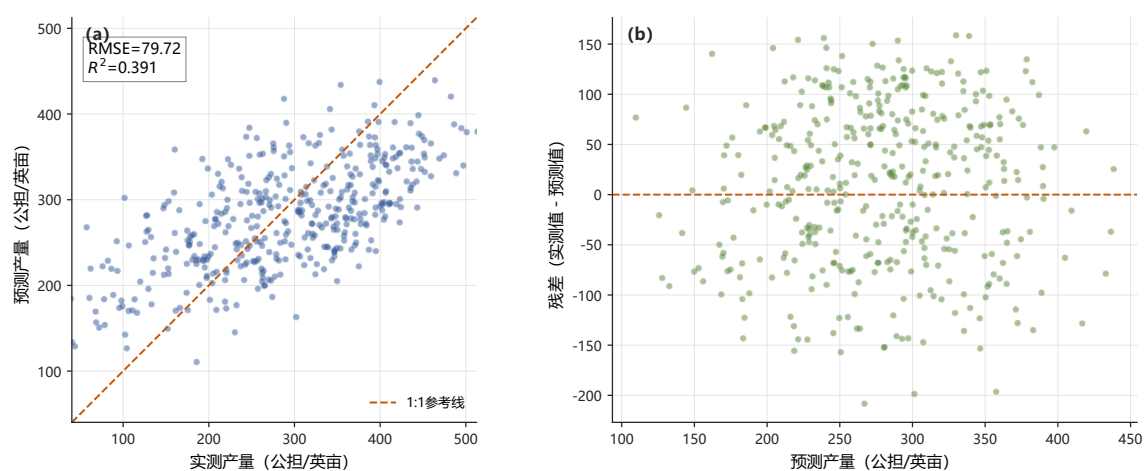


图 4 Elastic Net 在 2023 年时间外验证集上的预测与残差诊断

四、 问题二：水肥预算约束下的产量优化

4.1、 典型地块与可行域

直接把连续变量取平均、分类变量取众数，可能拼成现实中不存在的地块。本文在 2018—2022 年训练集中计算 Gower 混合距离 [4]，选取到全体样本平均距离最小的真实记录作为代表地块，得到原始索引 335。除灌溉量 W 和施肥量 F 外，气候、土壤、品种与管理字段均固定为该记录。

为了避免模型外推，以训练期 5%–95% 分位构造支持域：

$$572.938 \leq W \leq 1917.530, \quad 62.886 \leq F \leq 288.364. \quad (4.1)$$

水肥成本约束为

$$C(W, F) = 2.47105(p_w W + p_f F) \leq B, \quad (4.2)$$

基准取 $p_w = 0.20$ 元/ m^3 、 $p_f = 3.00$ 元/kg、 $B = 2500$ 元/公顷。水价参考地方经济作物农业用水公开标准 [5]；肥价敏感性端点参考 2026 年公开监测的尿素与磷酸二铵价格量级 [6]。水肥价格的地区差异通过敏感性表吸收，而不把单一价格写成普遍定价。

最终优化模型为

$$\begin{aligned} \max_{W, F} \quad & \hat{Y}(W, F \mid \mathbf{x}_0), \\ \text{s.t.} \quad & C(W, F) \leq 2500, \\ & W \in [572.938, 1917.530], \\ & F \in [62.886, 288.364]. \end{aligned} \quad (4.3)$$

4.2、 算法设计

由于只有两个连续决策变量，采用可审计的全域搜索而不是直接依赖随机智能算法。第一层在 101×101 网格上筛选可行点；第二层在 301×301 网格上加密，并从网格最优点进行局部无导数搜索；第三层对训练年份分块重采样 20 次，分别拟合 CatBoost 响应模型，以候选点预测分布的 10% 分位数作为保守目标：

$$(W_R, F_R) = \arg \max_{W, F} Q_{0.10} \{ \hat{Y}^{(b)}(W, F) : b = 1, \dots, 20 \}. \quad (4.4)$$

密网格同时承担全局覆盖和独立复算基准。模拟退火没有作为主算法，是因为二维规则可行域可以确定性覆盖；此时增加随机搜索并不能提供更强的最优性证据。

4.3、 最优候选与成本复算

101×101 网格和 301×301 加局部搜索得到完全相同的边界候选：

$$W^* = 1917.530 \text{ m}^3/\text{acre}, \quad F^* = 62.886 \text{ kg/acre}. \quad (4.5)$$

换算后 $W_{ha}^* = 4738.319 \text{ m}^3/\text{公顷}$, $F_{ha}^* = 155.395 \text{ kg/公顷}$ 。预测产量为 250.262 公担/英亩，成本为

$$C^* = 0.20 \times 4738.319 + 3.00 \times 155.395 = 1413.85 \text{ 元/公顷}, \quad (4.6)$$

预算余量 1086.15 元/公顷。独立程序重新读盘后得到的预测差和成本差均为 0，且约束违反量为 0。

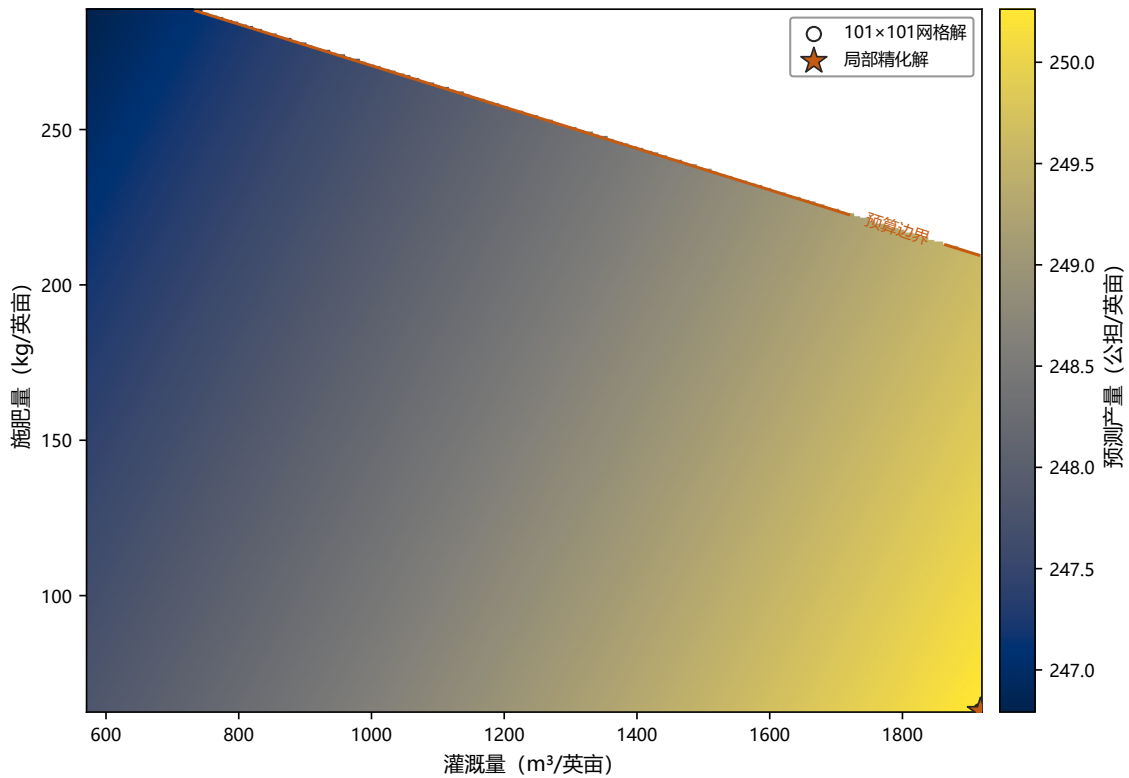


图 5 典型地块预算可行域内的水肥响应面（橙线为预算边界，星号为局部精化解）

20 模型下置 10% 分位鲁棒方案为 $W_R = 1854.782 \text{ m}^3/\text{英亩}$ 、 $F_R = 64.389 \text{ kg/英亩}$ ，

重采样预测均值 284.184、10% 分位 270.536、标准差 16.173，参考成本 1393.98 元/公顷。然而原选定 Elastic Net 在该点的预测为 250.142，两类模型相差 34.043 公担/英亩，表明算法自身已经收敛，但响应模型结构并不稳定。

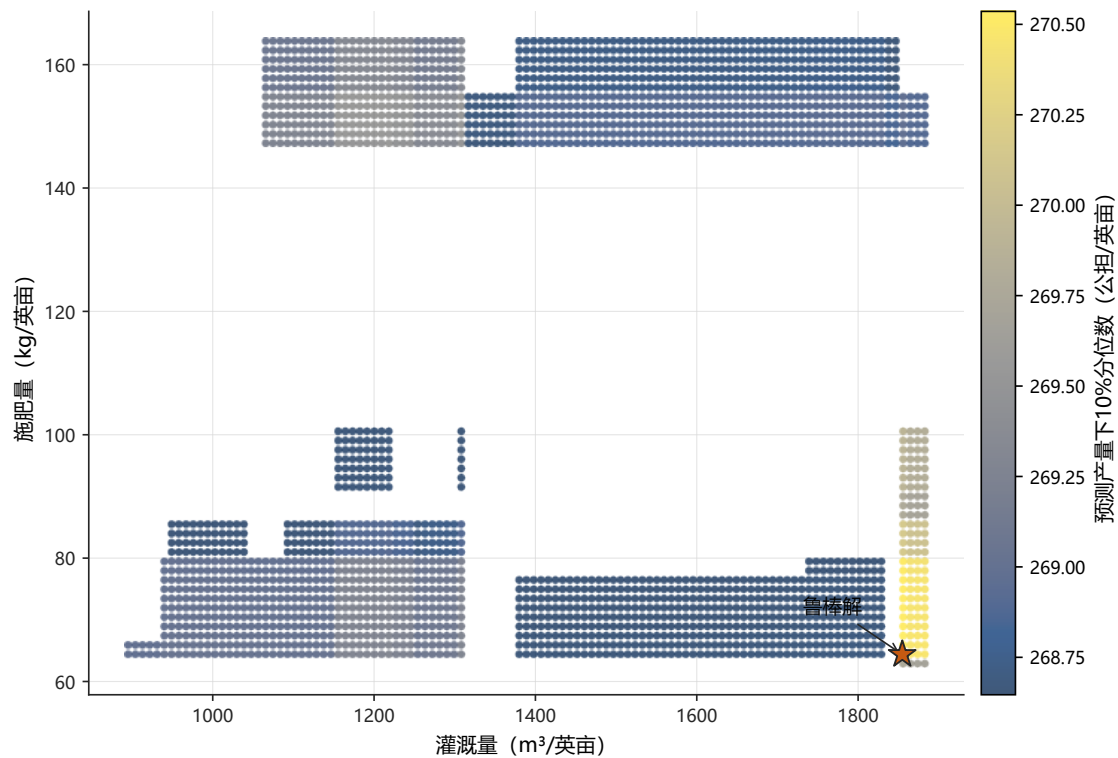


图 6 20 模型 Bootstrap 候选的产量下 10% 分位数（星号为鲁棒解）

4.4、 敏感性与结论边界

在水价 0.04、0.20、0.60 元/m³，肥价 2.19、3.00、4.63 元/kg，预算 2000、2500、3000 元/公顷的 27 种组合中，低、中水价下同一边界候选均可行；当水价为 0.60 元/m³时，预算成为有效约束，水量随预算与肥价在 859.784–1792.034 m³/英亩间变化，而施肥仍处下界。说明建议水量对水价敏感，不能脱离当地价格使用。

更关键的是，20 个重采样模型中“增加水量使预测上升”的比例只有 55%，“增加肥量使预测上升”的比例只有 40%。主解又同时命中水量上界和施肥量下界，故本文将 其定性为**支持域内的边界候选**。实际应用应先在 1855–1918 m³/英亩、63–65 kg/英亩的小范围设置田间试验，确认单位和边际效应后再做生产放大。

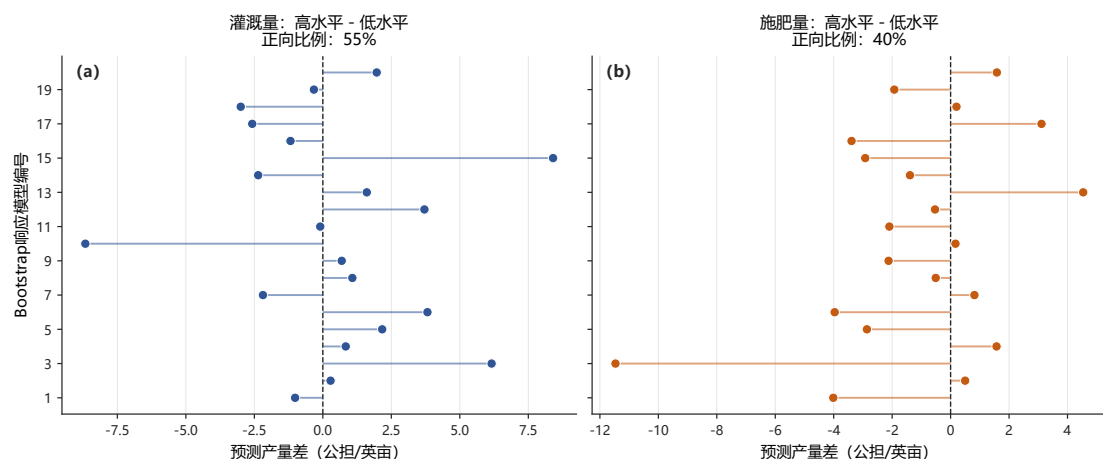


图 7 水肥边际效应在 20 个 Bootstrap 响应模型中的方向稳定性

五、 问题三：气候不确定下的鲁棒种植决策

5.1、 三类联合气候情景

选取平均温度、季节降雨、湿度、太阳辐射、蒸散量、热胁迫天数和霜冻天数构成气候向量。为降低极端值影响，对每个变量作稳健标准化

$$z_j = \frac{x_j - \text{median}(x_j)}{Q_{0.75}(x_j) - Q_{0.25}(x_j)}, \quad (5.1)$$

再以 PCA 保留至少 90% 累计方差，最后使用 PAM 型 K-medoids 分为 3 类。K-medoids 以真实记录为中心 [7]，能够保持温度、降雨与蒸散之间的实际联合关系。情景概率用各簇样本比例估计。表 4 中缺失的中心字段由训练期中位数补足，仅用于响应模型输入。

表 4 三个典型气候情景及关键变量

s	情景解释	π_s	平均温度 °C	季节降雨 mm	湿度 %	太阳辐射 MJ/(m ² d)	蒸散量 mm/d	热胁迫日 d	霜冻日 d
1	较高季雨、中等温度	0.4388	26.504	823.391	69.691*	23.758	6.114	20	4
2	较低季雨、偏凉	0.2878	22.627	697.816	67.521	23.411	4.668	7	7
3	中等季雨、偏热	0.2734	39.556	710.193	67.452	22.552*	4.633	17	4

注：* 表示原中心记录缺失，按 2018—2022 训练期中位数插补。

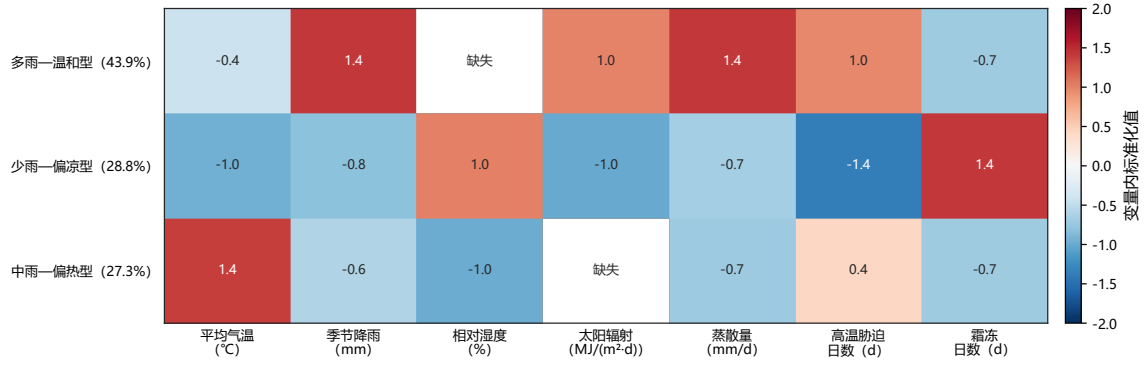


图 8 三个真实中心气候情景的变量内标准化特征（括号内为情景概率）

5.2、 max-min 模型与求解

固定问题二典型地块的土壤和其他管理条件，并将肥量固定为 62.886 kg/英亩。对三个品种和 501 个支持域水量点完全枚举，构造

$$\max_{v \in \mathcal{V}, W \in \mathcal{W}} \min_{s=1,2,3} \hat{Y}(v, W; \mathbf{c}_s, \mathbf{x}_0). \quad (5.2)$$

作为对照，同时计算概率加权期望最优方案

$$\max_{v, W} \sum_{s=1}^3 \pi_s \hat{Y}(v, W; \mathbf{c}_s, \mathbf{x}_0). \quad (5.3)$$

枚举保证离散品种被完整比较，一维水量网格则可直接检查全域，不依赖随机初值。稳健优化的含义是保护最差情景，而不是假设未来一定落在某一个情景 [8]。

5.3、 结果与鲁棒代价

选定 Elastic Net 下，max-min 与期望值准则得到同一方案：品种 Co98014，灌溉量 1917.530 m³/英亩。三个情景的预测产量依次为 253.376、266.704 和 291.580 公担/英亩，最差情景产量 253.376，概率加权期望 267.656。相对于期望值最优方案，鲁棒代价为 0，最差情景增益也为 0。该结果只表示当前响应面上两个准则的最优点重合，不是 max-min 算法的普遍性质。

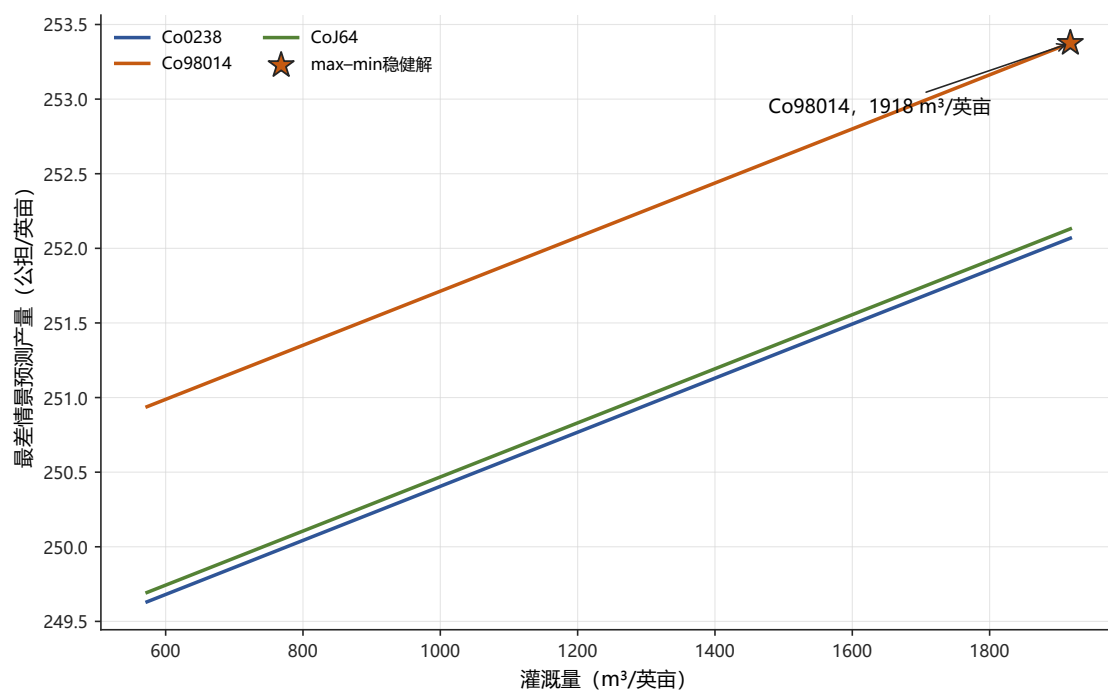


图 9 各品种在三情景下的最差产量（星号为 max-min 稳健解）

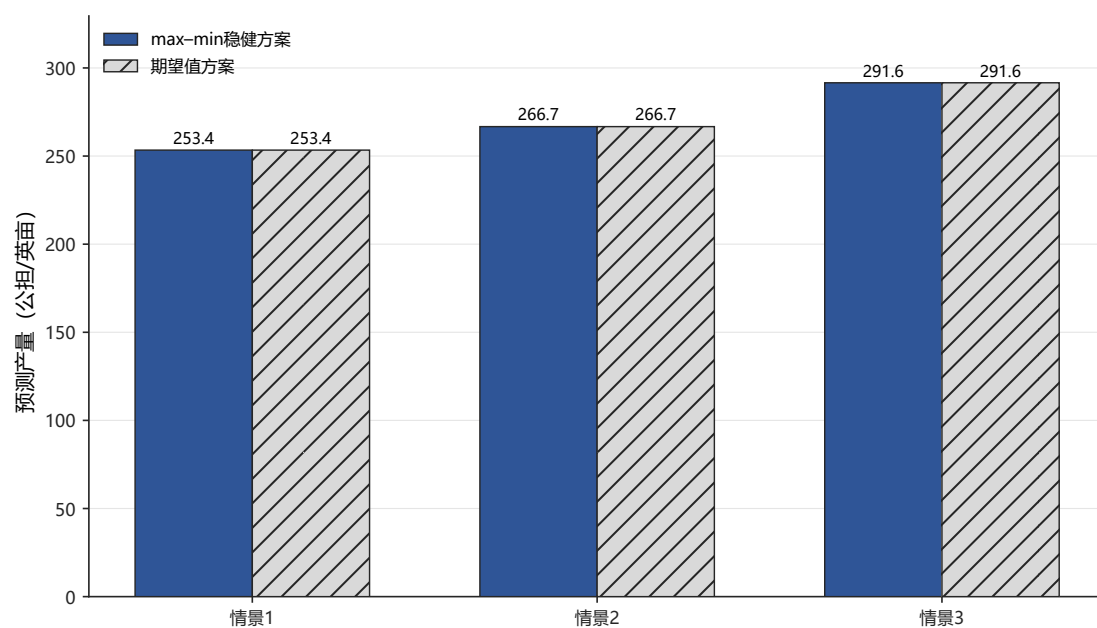


图 10 max-min 方案与期望值方案在三个情景下的预测产量

5.4、 情景和响应模型稳定性

对气候中心进行 100 次分块重采样时，在同一 Elastic Net 下 Co98014 选择率为 100%，水量区间固定在 1917.530，说明情景抽样扰动不会改变该线性响应面的排序。但把响应模型也纳入 20 次重采样后，稳健品种选择率变为 Co0238 占 50%、Co98014 占

35%、CoJ64 占 15%，水量 95% 范围为 572.938–1895.792 m³/英亩。由此可见，不确定性的主要来源不是三簇气候中心，而是观察数据所能支持的水肥—品种响应结构。

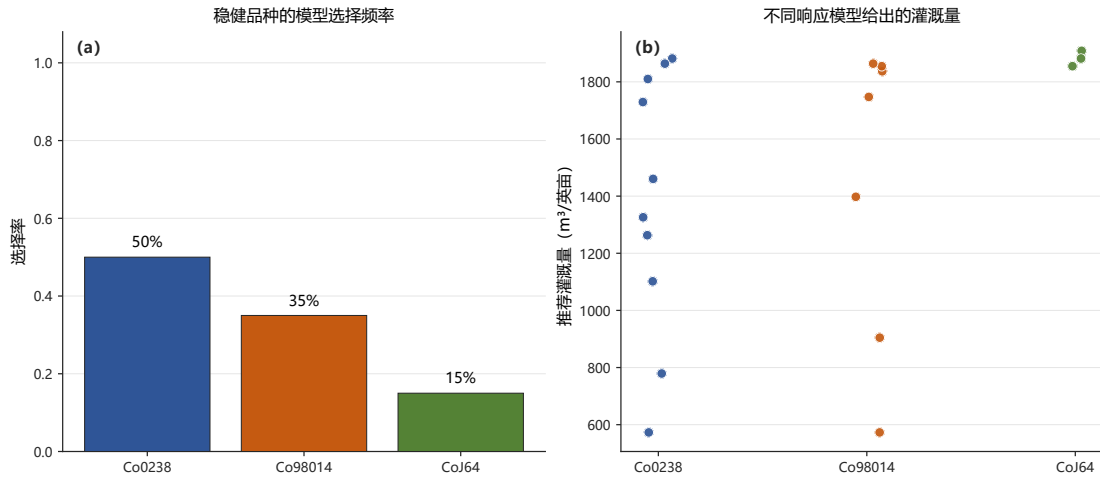


图 11 20 个 Bootstrap 响应模型下的品种选择率与推荐灌溉量

因此，Co98014 是**选定模型下的稳健候选**，不能称为唯一稳健品种。若用于实地决策，应将 Co0238 一并纳入小区试验，并优先收集水量单位、品种与水分交互的独立数据。

六、 问题四：产量与环境压力的多目标权衡

6.1、 环境压力代理量

附件没有氮淋失、抽水能耗、碳排放或污染治理成本的实测值。若直接给水肥乘以任意排放系数，会制造无法核验的“环境成本”。因此先把水肥量按训练期 5%–95% 分位归一化并截断：

$$\tilde{W} = \text{clip} \left(\frac{W - W_{0.05}}{W_{0.95} - W_{0.05}}, 0, 1 \right), \quad \tilde{F} = \text{clip} \left(\frac{F - F_{0.05}}{F_{0.95} - F_{0.05}}, 0, 1 \right), \quad (6.1)$$

构造无量纲压力

$$E(W, F) = \lambda_w \tilde{W}^2 + (1 - \lambda_w) \tilde{F}^2, \quad \lambda_w = 0.5. \quad (6.2)$$

平方项表示高投入阶段的边际压力加速，但 E 不等同于真实污染量或人民币环境损失。问题四的目标已由经济预算转为环境压力，因此主模型不再施加问题二的预算，只保留

观测支持域；经济成本仅作为参考量报告。

6.2、 Pareto 前沿与算法

双目标写为

$$\max \hat{Y}(W, F), \quad \min E(W, F). \quad (6.3)$$

在水肥密集可行网格上，依次给定环境上限 ε_k 并求

$$\max_{W, F} \hat{Y}(W, F) \quad \text{s.t.} \quad E(W, F) \leq \varepsilon_k, \quad (6.4)$$

再删除被严格支配的重复点，得到增广 ε 约束式前沿 [9]。该方法不会像单一加权和那样遗漏非凸部分。高阶对照采用 NSGA-II 的非支配排序、拥挤度和交叉变异机制 [10]，只用于检查前沿搜索质量，不替换可复算的密网格。

共得到 351 个 Pareto 点。为透明选择折中点，分别对产量损失 $L = Y_{\max} - \hat{Y}$ 和环境压力作 min-max 归一化，选择到理想点 $(0, 0)$ 欧氏距离最小者：

$$i^* = \arg \min_i \sqrt{\tilde{L}_i^2 + \tilde{E}_i^2}. \quad (6.5)$$

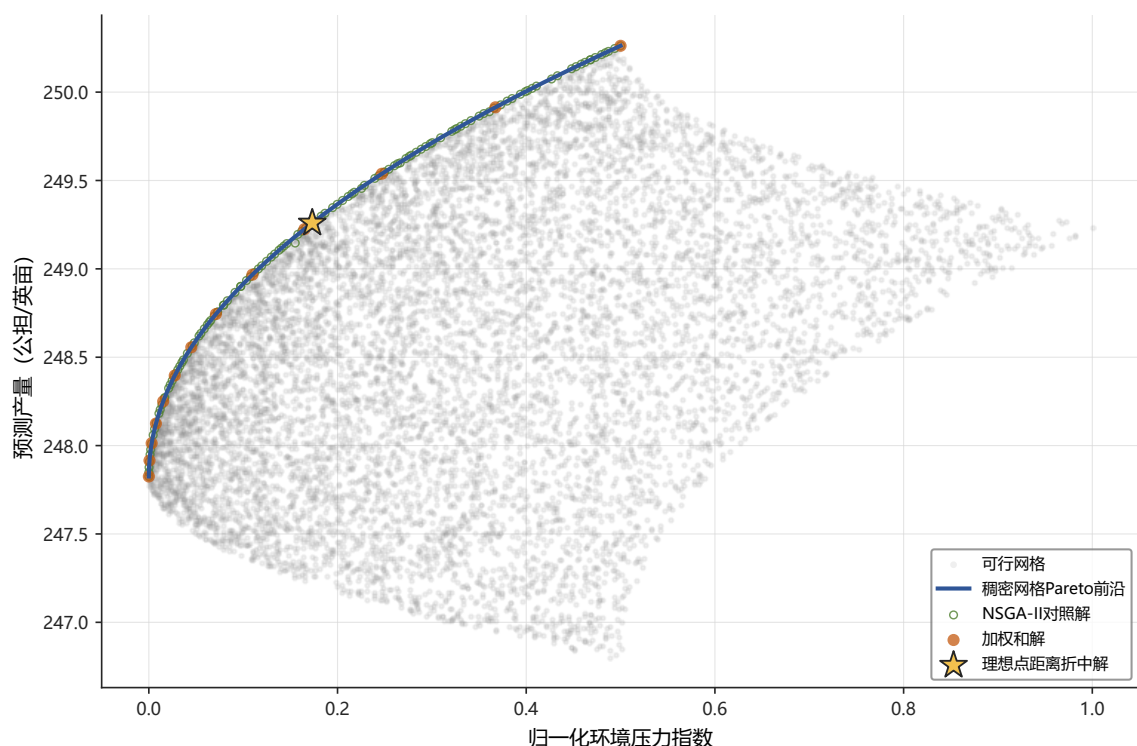


图 12 产量—环境压力 Pareto 前沿及三类算法对照（星号为理想点距离折中解）

6.3、折中解与邻域替代方案

平衡点为

$$W^* = 1364.326 \text{ m}^3/\text{acre}, \quad F^* = 62.886 \text{ kg}/\text{acre}, \quad (6.6)$$

预测产量 249.260 公担/英亩，环境压力 0.173208，参考经济成本 1140.45 元/公顷。与问题二的最大产量候选相比，灌溉量减少 553.204 m^3 /英亩，而产量只减少 1.003 公担/英亩，显示在选定模型中，高水量末端的产量增益较小。独立复算表明产量与环境压力差均为 0，且不存在严格支配该点的网格解。

折中点附近的五个可执行方案如表 5。这组结果表明，决策误差在数个 m^3 /英亩范围内不会造成突变，实际执行可按灌溉设施精度取整。

表 5 折中点附近的 Pareto 替代方案

$W/(m^3/\text{英亩})$	$F/(kg/\text{英亩})$	预测产量	环境压力	参考成本
1356.643	62.886	249.246	0.169861	1136.65
1360.484	62.886	249.253	0.171531	1138.55
1364.326	62.886	249.260	0.173208	1140.45
1368.168	62.886	249.267	0.174894	1142.35
1372.009	62.886	249.274	0.176588	1144.25

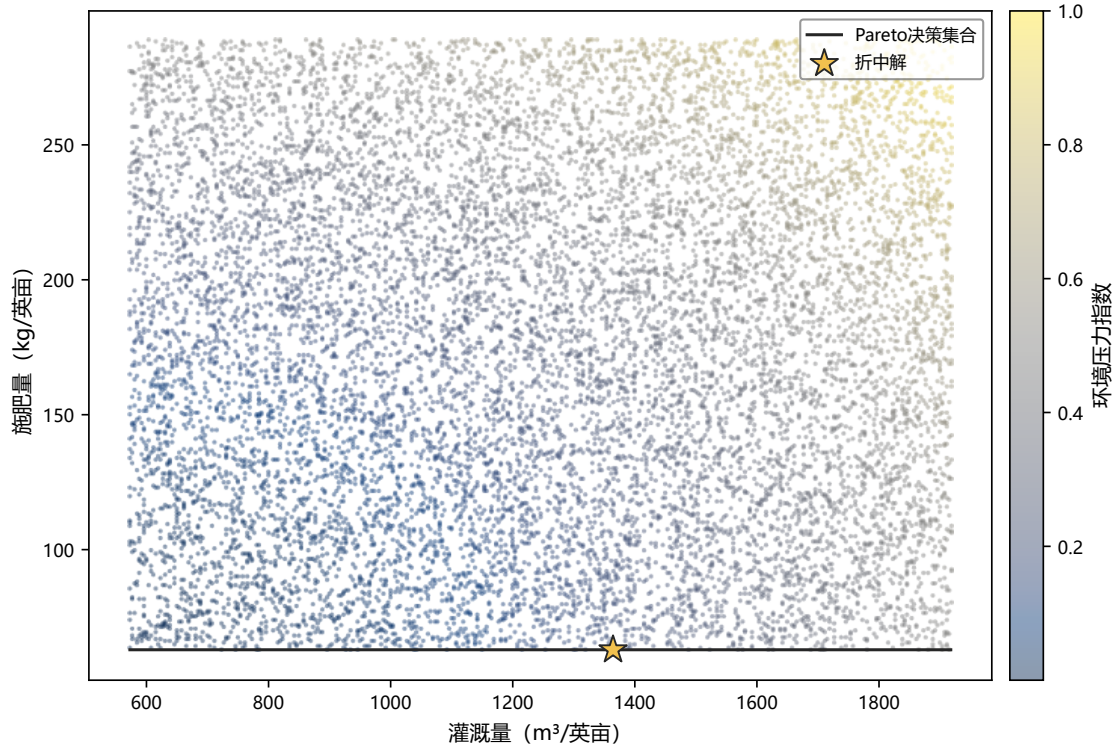


图 13 水肥决策空间中的可行网格、Pareto 决策集合与折中点

6.4、多重稳健性检验

当 λ_w 在 0.3、0.5、0.7 间变化时，单独归一化后的折中点均保持在 $W = 1364.326$ 、 $F = 62.886$ ，说明等权结论不依赖该三档权重。NSGA-II 候选到密网格前沿的归一化中位距离为 0.001169、最大距离 0.011306，说明随机算法与确定性前沿基本一致。

然而，响应模型 Bootstrap 下折中点水量 95% 区间为 572.938–1291.174，肥量区间为 62.886–220.814，宽度较大。故图 12 的前沿在选定 *Elastic Net* 下得到可靠复算，而

不同响应模型下的农业推荐仍有显著不确定性。可执行建议应优先采用折中点附近的低肥方案，并通过田间数据重新标定环境代理量。

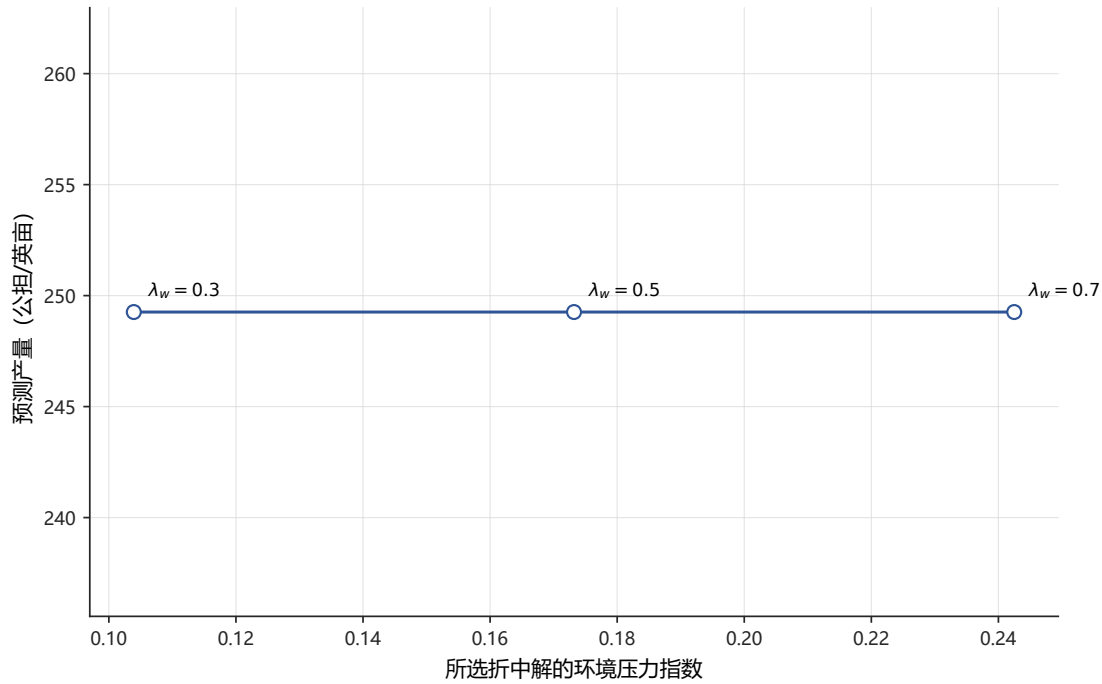


图 14 环境权重变化下折中点的环境压力与预测产量

七、 问题五：品种—季节适应性综合评价

7.1、 协变量校正的混合效应模型

数据中三个品种与 Kharif、Rabi 两季构成 6 个完整组合，每组样本数为 416–506。简单均值会混合不同地块的气候、土壤和管理条件，因此以“品种 × 季节”为固定效应，加入 18 个标准化协变量，以年份和农业记录簇为随机效应：

$$Y_i = \beta_0 + \beta_{v(i),r(i)} + \gamma^\top z_i + u_{\text{year}(i)} + u_{\text{cluster}(i)} + \varepsilon_i, \quad (7.1)$$

其中 $u_{\text{year}} \sim N(0, \sigma_y^2)$, $u_{\text{cluster}} \sim N(0, \sigma_c^2)$ 。混合模型可以在非平衡多环境数据中分离组合平均差异与组间波动 [11]。模型用 2747 行可用记录拟合并收敛，但 Hessian 非正定，故固定效应标准误和置信区间只作参考，不能据此宣称品种处理的因果差异。

通过把每个组合放到相同协变量分布上预测，得到调整均值、调整 10% 分位和组间稳定性。六组的调整均值仅为 274.330–285.622，且区间高度重叠；这也说明只按均值排列会产生过度确定的名次。

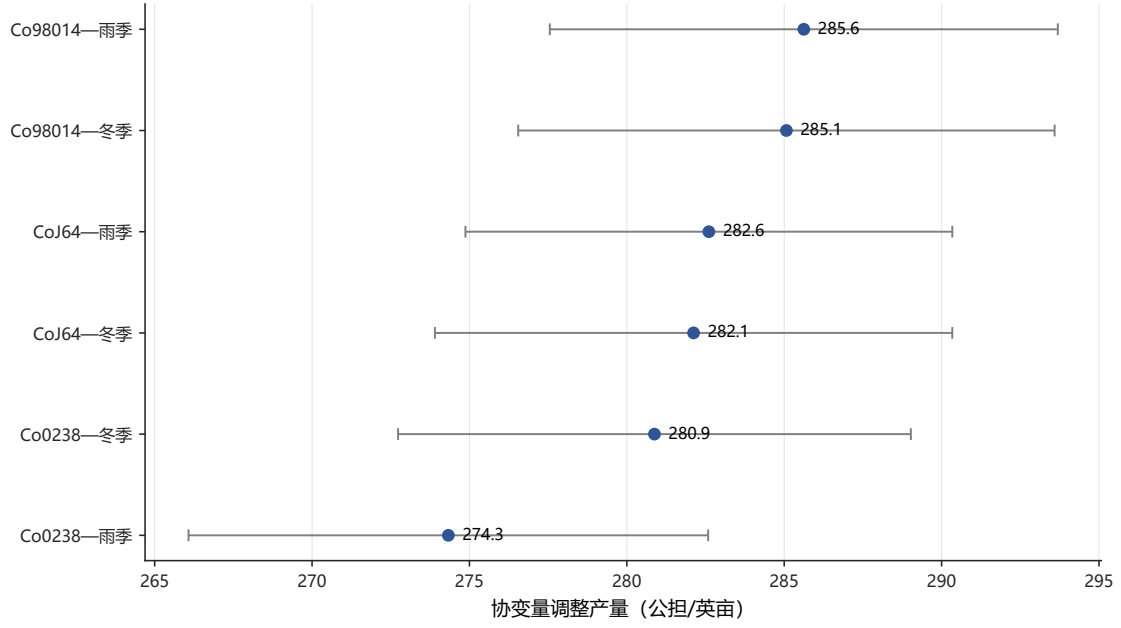


图 15 六个品种—季节组合的协变量调整均值及 95% 置信区间

7.2、 CRITIC 赋权与 TOPSIS 排序

候选指标包括调整均值、调整 10% 分位、稳定性分数和最差情景相对表现。先检查相关性并删除高度冗余的最差情景相对表现，保留前三项。对正向化并标准化后的矩阵 $Z = (z_{ij})$ ，CRITIC 信息量定义为 [12]

$$C_j = s_j \sum_{k=1}^m (1 - r_{jk}), \quad \omega_j = \frac{C_j}{\sum_k C_k}, \quad (7.2)$$

其中 s_j 表示对比强度， r_{jk} 表示指标间相关系数。得到调整均值、稳定性和调整 10% 分位的权重依次为 0.3978、0.3632、0.2390。

TOPSIS 按加权欧氏距离计算 [13]

$$D_i^+ = \sqrt{\sum_j \omega_j (z_{ij} - z_j^+)^2}, \quad D_i^- = \sqrt{\sum_j \omega_j (z_{ij} - z_j^-)^2}, \quad S_i = \frac{D_i^-}{D_i^+ + D_i^-}. \quad (7.3)$$

S_i 越大，方案越接近正理想点。该方法同时保留高产、下尾保护和跨组稳定性，而不会让单一均值支配排名。

表 6 综合评价前三个候选及排名概率

名义名次	组合	TOPSIS 分数	调整均值	Top-3 概率	第一概率
1	CoJ64-Kharif	0.7167	282.606	0.612	0.292
2	CoJ64-Rabi	0.6021	282.120	0.532	0.184
3	Co0238-Rabi	0.5871	280.878	0.496	0.216

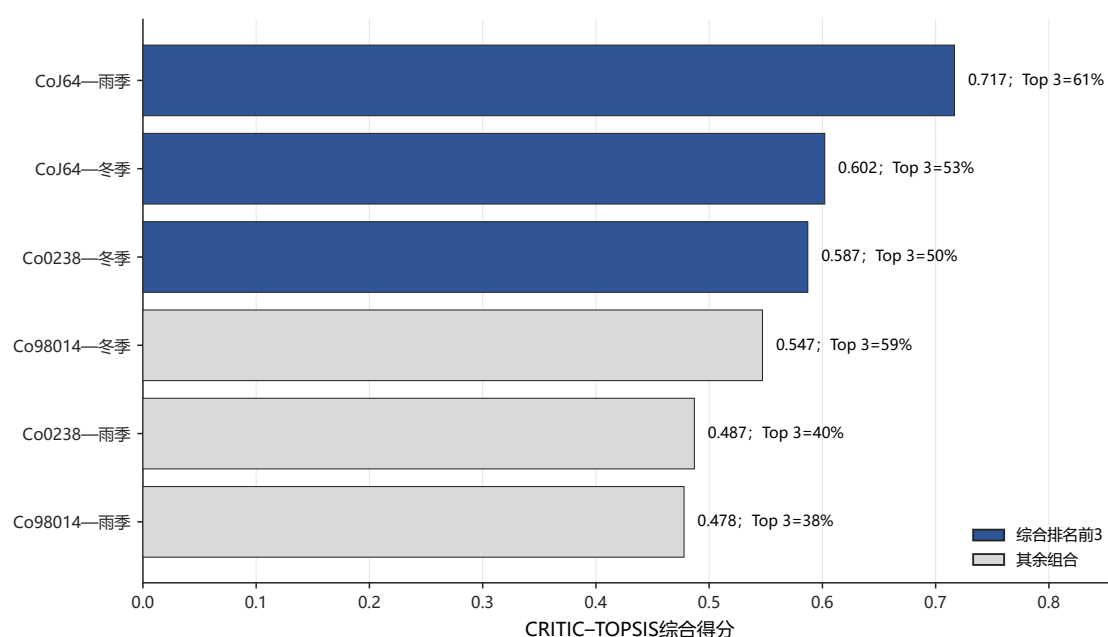


图 16 CRITIC-TOPSIS 综合得分及 500 次重采样的 Top 3 概率

500 次年份与样本重采样中，三个名义前三组合的 Top-3 概率均低于 70%，第一名概率也只有 18.4%–29.2%。因此它们应作为**并列优先候选集**，而不是确定性的第一、第二、第三。Co98014 两季虽然调整均值最高，但下尾表现或稳定性较弱，未进入综合前三，恰好说明综合评价不同于均值排序。

7.3、 第一候选的适宜条件

CoJ64-Kharif 共有 506 条记录。其观测条件的均值和四分位范围为：平均温度 27.890°C (22.068–34.351°C)，季节降雨 792.104 mm (568.458–999.881 mm)，湿度 70.287% (59.709%–80.775%)，太阳辐射 22.897 MJ/(m²d) (18.982–27.111)，土壤 pH 7.269 (6.620–7.924)，土壤含水率 25.546% (18.333%–33.014%)，有机碳 0.824% (0.490%–1.163%)。氮、

磷、钾的四分位范围分别约为 100.713–200.867、46.232–94.897 和 71.843–158.032 kg/英亩。

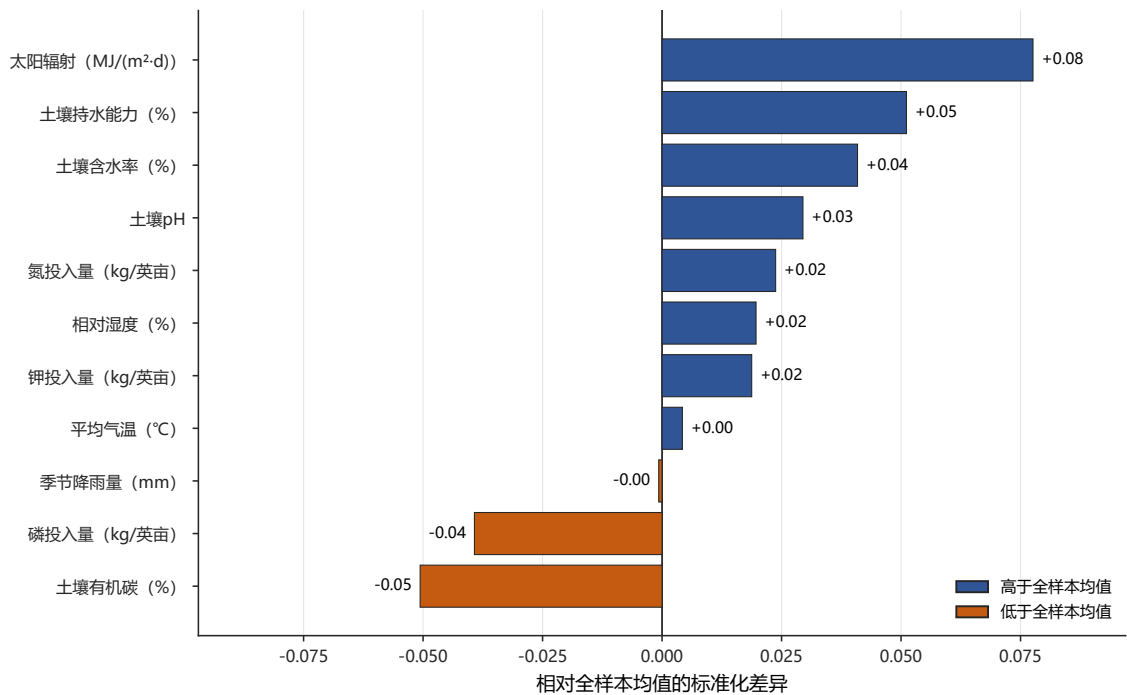


图 17 CoJ64-Kharif 观测条件相对全样本均值的标准化差异

图 17 显示各标准化差异绝对值均小于 0.08，说明第一候选并不依赖一个极端狭窄的气候或土壤区间。上述范围应理解为“样本中表现较好的常见支持域”，不是严格适生阈值。若用于新品种布局，应保留 CoJ64 两季和 Co0238-Rabi 三个候选，并在不同地块分层试种后更新排名概率。

八、 结果分析与模型检验

8.1、 统一验证协议

模型检验不以“程序成功运行”为终点，而按数据、预测、优化、排名和图件五层进行。首先对原始 CSV 计算 SHA-256，建模前后哈希完全一致：

DD8D3B5A4BBD2D9DB659C2C2FCBF7442B1416E09921B674CB7046E8A659B615A

从而确认原数据未被覆盖。其次，对问题一使用两个连续外年验证集；对问题二至四另写复算逻辑，不复用优化器内存中的目标值；对问题五检查六个组合是否各出现一次，

并验证 500 次重采样的每一次都形成完整排名。最后检查正文每幅分析图是否同时存在 PNG 和矢量 PDF，且 PNG 不低于 2000×1100 像素。

独立验收共 27 项，结果为 23 项通过、4 项警告、0 项失败。警告并不是程序错误，而是对结论不确定性的主动保留，具体为：问题二候选触及观测支持边界；水肥效应方向在 20 个响应模型中不稳定；问题三品种选择随响应模型变化；问题五前三组合的 Top-3 概率均低于 70%。

表 7 核心结论的验证等级与使用方式

问题	核心结论	证据等级	使用边界
一	Elastic Net 具中等外年预测力	已验证	可作支持域内响应模型；不可作因果解释
二	水 1917.530、肥 62.886 的边界候选	暂定	需确认单位并以小范围田间试验校准
三	Co98014 为选定模型下稳健候选	暂定	响应模型重采样中 Co0238 选择率更高
四	水 1364.326、肥 62.886 在选定模型前沿上	有局限支持	环境指标为代理量，Bootstrap 区间宽
五	三个组合构成并列优先候选集	有局限支持	名义次序不稳定，不作确定名次承诺

8.2、 量纲、极端条件与边界检验

问题二的成本统一在公顷尺度计算，水量和肥量先乘 2.47105，避免把英亩投入直接代入元/公顷预算。若 $W = F = 0$ ，环境压力式给出 0；若二者均位于支持域上界，则 $E = 1$ ，满足极端值解释。问题三中三情景概率之和为 1.000000，最差产量通过逐行取最小值独立重算，最大差为 0。问题四全部 351 个保存前沿点均不存在严格支配者，折中点的严格支配者数为 0。

响应模型不允许水肥越过训练期 5%–95% 分位。该约束降低了外推风险，却也使问题二和三的高水量候选落在上边界，无法判断真正峰值是否在域外。因此“边界最优”

只能说明观测域内目标仍在上升，不能解释为物理最适灌溉量。

8.3、 算法对照与随机稳定性

问题二的 101×101 网格、 301×301 网格和局部无导数搜索返回同一点，证明数值搜索本身稳定。问题四的确定性密网格与 NSGA-II 前沿中位归一化距离 0.001169、最大 0.011306，支持 Pareto 结构不是网格分辨率的偶然产物。所有随机过程使用种子 20260728，并保存逐次重采样表，而不是只保存最后一张图。

问题三明确区分两类不确定性：情景中心重采样保持 Co98014 不变，但响应模型重采样使三个品种均可能被选中。问题五同样用概率而非单次名次表达不确定性。这两项检验说明，不应以一次优化结果或一条排序曲线替代模型结构风险。

8.4、 论文、代码和图表一致性

正文数值全部来自 `results/q1` 至 `results/q5` 下的 CSV 或 JSON；表格中的主要数字由相同结果文件转写，图件由对应问题程序生成。源程序编译检查无语法错误；17 幅分析图均有 PNG 与矢量 PDF 配对，正文使用 PDF 版本以保证缩放质量。支撑材料中的 `verify_gate3.py` 可在不访问训练过程内存对象的情况下重算哈希、约束、情景、Pareto 非支配性和排名完整性。

九、 模型评价与改进方向

9.1、 模型优点

第一，时间切分与决策安全特征共同控制了最常见的数据泄漏，2024 年只在模型选择冻结后使用。第二，建立了“简单模型—主模型—高阶模型”的可淘汰比较，复杂算法没有因形式新颖而自动胜出。第三，优化结果均有支持域、单位、成本和独立复算证据，问题四还用确定性前沿与随机算法交叉验证。第四，模型没有隐藏负面结果：水肥效应不稳定、边界命中、混合模型 Hessian 警告和排名低概率均写入正文。第五，原始数据、派生数据、程序、结果、图件和论文之间保留可追踪文件关系。

9.2、 模型局限

其一，记录级气候字段并非真实逐月面板，无法识别关键生育阶段的水热胁迫。其二，水量字段的物理单位由量级解释而非元数据确认；若实际含义不同，问题二至四的绝对推荐必须重算。其三， R^2 约 0.39 表明产量还受未记录的耕作、测量误差和地块历史影响。其四，环境压力只反映相对投入，未包含氮淋失、抽水能耗和污染受体。其五，观察数据存在管理者根据地块状况调整水肥的反向因果，当前模型不能给出严格的剂量因果效应。其六，问题五 MixedLM 虽收敛但 Hessian 非正定，名次概率也偏低。

9.3、 可实施改进

后续应优先收集同一地块的逐月或生育阶段气候、灌溉事件、水肥配方、土壤氮动态和重复产量；通过随机区组或分层小区试验标定水肥响应。若获得氮淋失和能耗数据，可把无量纲 E 替换为物理环境代价，并对 Pareto 前沿重新计算。对问题三，可用多季气候预报构建分布鲁棒或 CVaR 模型；对问题五，可增加地点—年份—品种交互并采用贝叶斯层次模型，使用后验预测检查和排名概率替代不稳定的点估计。

在现有证据下，最可信的成果是问题一的外年响应能力和问题四在选定模型下的前沿结构；问题二、三的单点建议与问题五的名义次序必须配合区间、候选集和试验验证使用。

参考文献

- [1] ZOU H, HASTIE T. Regularization and variable selection via the elastic net[J]. Journal of the Royal Statistical Society: Series B, 2005, 67(2): 301–320.
- [2] PROKHORENKOVA L, GUSEV G, VOROBIEV A, et al. CatBoost: unbiased boosting with categorical features[C]//Advances in Neural Information Processing Systems 31. Montreal: Curran Associates, 2018: 6638–6648.
- [3] FOOD AND AGRICULTURE ORGANIZATION OF THE UNITED NATIONS. Sugarcane: crop water information[EB/OL]. [2026-07-28]. <https://www.fao.org/land-water/databases-and-software/crop-information/sugarcane/>.

- [4] GOWER J C. A general coefficient of similarity and some of its properties[J]. *Biometrics*, 1971, 27(4): 857–871.
- [5] 云阳县发展和改革委员会, 云阳县水利局. 关于制定我县农业用水指导价格的通知 [EB/OL]. [2026-07-28]. https://www.yunyang.gov.cn/bm_257/xfzggw/zwgk_62088/jczfxxgk/zdxxgs/202601/t20260118_15330102.html.
- [6] 中国价格信息网. 2026 年 5 月全国农资价格行情表 [EB/OL]. [2026-07-28]. <https://www.chinaprice.cn/jsdzqk/60591.jhtml>.
- [7] KAUFMAN L, ROUSSEEUW P J. Finding Groups in Data: An Introduction to Cluster Analysis[M]. New York: Wiley, 1990.
- [8] BERTSIMAS D, SIM M. The price of robustness[J]. *Operations Research*, 2004, 52(1): 35–53.
- [9] MAVROTAS G. Effective implementation of the ε -constraint method in multi-objective mathematical programming problems[J]. *Applied Mathematics and Computation*, 2009, 213(2): 455–465.
- [10] DEB K, PRATAP A, AGARWAL S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. *IEEE Transactions on Evolutionary Computation*, 2002, 6(2): 182–197.
- [11] PINHEIRO J C, BATES D M. Mixed-Effects Models in S and S-PLUS[M]. New York: Springer, 2000.
- [12] DIAKOULAKI D, MAVROTAS G, PAPAYANNAKIS L. Determining objective weights in multiple criteria problems: the CRITIC method[J]. *Computers & Operations Research*, 1995, 22(7): 763–770.
- [13] HWANG C L, YOON K. Multiple Attribute Decision Making: Methods and Applications[M]. Berlin: Springer-Verlag, 1981.

附录 A 支撑材料文件列表与复现说明

表 8 支撑材料目录说明

目录或文件	内容与用途
README.txt	运行顺序、Python 版本、随机种子、输入哈希及复现注意事项。
requirements.txt	Python 依赖及经验证版本。
code/	本文全部完整 Python 源程序，包括数据审计、清洗、五问建模、绘图、结果刷新与独立验收。
data/derived/	清洗日志和未插补建模框架；为自主生成的派生数据，不含题目原始 CSV。
results/q1--q5/	模型指标、预测、优化网格摘要、情景、Pareto 前沿、排名与重采样结果。
results/final_verification_*	27 项独立验收的机器可读结果与汇总。
figures/	正文 17 幅分析图的 PNG 与矢量 PDF 版本，均由源程序生成。
paper/	最终论文 PDF 及完整 Overleaf/LaTeX 工程。

推荐运行环境为 Python 3.12.13。主依赖版本为 NumPy 2.3.5、pandas 3.0.1、SciPy 1.18.0、scikit-learn 1.9.0、statsmodels 0.14.6、interpret 0.7.8、CatBoost 1.2.10、SHAP 0.52.0、pymoo 0.6.2、Matplotlib 3.11.1、Seaborn 0.13.2 和 joblib 1.5.3。随机种子统一为 20260728。

表 9 源程序文件清单

程序文件	主要用途
stage1_audit.py	原始数据质量审计、异常检查与审计图生成。
data_pipeline.py	数据读取、变量角色控制、时间外推切分、缺失值处理与成本换算。
q1_models.py、 reselect_q1.py	候选产量响应模型、外年验证、简约规则选模、误差区间与解释分析。
q2_optimize.py	典型地块选择、水肥约束、网格搜索、局部精化及鲁棒最优解。
q3_robust.py	气候情景构造、品种-水分组合评价与 max-min 稳健决策。
q4_pareto.py	环境压力指标、非支配前沿、折中点选择及 NSGA-II 对照。
q5_adaptation.py	混合效应校正、跨情景相对表现与 CRITIC-TOPSIS 综合排序。
run_gate3.py、 resume_gate3.py	五问总控及断点恢复。
refresh_gate3_ reports.py、 refresh_selected_ figures.py	结果报告和论文图件刷新。

<code>verify_gate3.py</code>	输入哈希、约束、Pareto 前沿、排名和图件的一致性验收。
------------------------------	--------------------------------

附录 B 程序代码

B.1、 数据清洗与时间外推切分

Listing 1 `data_pipeline.py`: 数据解析、年度切分与缺失值处理核心程序

```

141 def parse_sunshine_hours(series: pd.Series) -> pd.Series:
142     """Convert HH:MM duration strings to decimal hours without guessing bad rows."""
143
144     text = series.astype("string")
145     parts = text.str.extract(r"^\s*(\d{1,2}):(\d{2})\s*$")
146     hours = pd.to_numeric(parts[0], errors="coerce")
147     minutes = pd.to_numeric(parts[1], errors="coerce")
148     valid = hours.between(0, 24) & minutes.between(0, 59)
149     result = hours + minutes / 60.0
150     return result.where(valid)
151
152
153 def load_raw(path: str | Path) -> pd.DataFrame:
154     """Read the immutable source and add only documented derived columns."""
155
156     df = pd.read_csv(path, encoding="utf-8-sig")
157     df["Sunshine_Hours_decimal"] = parse_sunshine_hours(df["Sunshine_Hours_hh_mm"])
158     df[YEARS] = pd.to_numeric(df[YEARS], errors="coerce")
159     return df
160
161
162 def model_frame(df: pd.DataFrame, features: Iterable[str]) -> pd.DataFrame:
163     """Return the requested model columns in a stable order."""
164
165     cols = [c for c in features if c in df.columns]
166     return df.loc[:, cols].copy()
167
168
169 def split_years(df: pd.DataFrame) -> dict[str, pd.DataFrame]:
170     """Primary leakage-safe split confirmed at Gate 1."""
171
172     return {
173         "train": df[df[YEARS].between(2018, 2022, inclusive="both")].copy(),
174         "validation": df[df[YEARS].eq(2023)].copy(),
175         "test": df[df[YEARS].eq(2024)].copy(),
176         "unknown_year": df[df[YEARS].isna()].copy(),
177     }
178
179
180 def expanding_year_folds(train_df: pd.DataFrame) -> list[tuple[np.ndarray, np.ndarray, int]]:
181     """Expanding-window folds: 2018-19->2020, then through 2022."""
182
183     folds: list[tuple[np.ndarray, np.ndarray, int]] = []
184     years = train_df[YEARS].to_numpy()
185     for validation_year in (2020, 2021, 2022):

```



```

186         tr = np.flatnonzero((years >= 2018) & (years < validation_year))
187         va = np.flatnonzero(years == validation_year)
188         if tr.size and va.size:
189             folds.append((tr, va, validation_year))
190     return folds
191
192
193 def numeric_and_categorical(df: pd.DataFrame, features: Iterable[str]) -> tuple[list[str], list[str]]:
194     cols = [c for c in features if c in df.columns]
195     numeric = [c for c in cols if pd.api.types.is_numeric_dtype(df[c])]
196     categorical = [c for c in cols if c not in numeric]
197     return numeric, categorical
198
199
200 @dataclass
201 class TabularFiller:
202     """Training-only imputation used by EBM and CatBoost."""
203
204     numeric: list[str]
205     categorical: list[str]
206     medians: dict[str, float]
207
208     @classmethod
209     def fit(cls, frame: pd.DataFrame, features: Iterable[str]) -> "TabularFiller":
210         numeric, categorical = numeric_and_categorical(frame, features)
211         medians = {
212             c: float(pd.to_numeric(frame[c], errors="coerce").median())
213             for c in numeric
214         }
215         return cls(numeric=numeric, categorical=categorical, medians=medians)
216
217     def transform(self, frame: pd.DataFrame) -> pd.DataFrame:
218         out = frame.loc[:, self.numeric + self.categorical].copy()
219         for c in self.numeric:
220             out[c] = pd.to_numeric(out[c], errors="coerce").fillna(self.medians[c]).astype(float)
221         for c in self.categorical:
222             out[c] = out[c].astype("string").fillna("__MISSING__").astype(str)
223         return out
224
225     @property
226     def ordered_features(self) -> list[str]:
227         return self.numeric + self.categorical
228
229     @property
230     def feature_types(self) -> list[str]:
231         return ["continuous"] * len(self.numeric) + ["nominal"] * len(self.categorical)

```

B.2、 问题一： 候选模型选择与误差检验

Listing 2 q1_models.py: Elastic Net 超参数的年度滚动验证

```

124 def tune_elastic(train: pd.DataFrame, features: list[str]) -> tuple[float, float, pd.DataFrame]:
125     base = elastic_pipeline(train, features, alpha=0.1, l1_ratio=0.5)
126     folds = [(tr, va) for tr, va, _ in expanding_year_folds(train)]
127     search = GridSearchCV(
128         base,
129         param_grid={
130             "model__alpha": [0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0],
131             "model__l1_ratio": [0.05, 0.25, 0.5, 0.75, 0.95],
132         },
133         scoring="neg_root_mean_squared_error",
134         cv=folds,
135         n_jobs=-1,

```

```

136         refit=False,
137         error_score="raise",
138     )
139     search.fit(train[features], train[TARGET])
140     result = pd.DataFrame(search.cv_results_).sort_values("rank_test_score")
141     best = result.iloc[0]
142     return (
143         float(best["param_model__alpha"]),
144         float(best["param_model__l1_ratio"]),
145         result,
146     )

```

Listing 3 q1_models.py: 候选模型外年验证与 Bootstrap 区间

```

204 def cross_validate_candidate(
205     candidate: str,
206     train: pd.DataFrame,
207     features: list[str],
208     elastic_params: tuple[float, float],
209 ) -> pd.DataFrame:
210     rows: list[dict] = []
211     for tr_idx, va_idx, year in expanding_year_folds(train):
212         tr, va = train.iloc[tr_idx], train.iloc[va_idx]
213         started = time.perf_counter()
214         if candidate == "Mean baseline":
215             fitted = fit_mean(tr, features)
216         elif candidate == "Elastic Net":
217             fitted = fit_elastic(tr, features, *elastic_params)
218         elif candidate == "EBM":
219             fitted = fit_ebm(tr, features)
220         elif candidate == "CatBoost":
221             fitted = fit_catboost(tr, features, iterations=600)
222         else:
223             raise ValueError(candidate)
224         metrics = regression_metrics(va[TARGET].to_numpy(), fitted.predict(va))
225         rows.append(
226             {
227                 "model": candidate,
228                 "validation_year": year,
229                 "train_n": len(tr),
230                 "validation_n": len(va),
231                 "runtime_seconds": time.perf_counter() - started,
232                 **metrics,
233             }
234         )
235     return pd.DataFrame(rows)
236
237
238 def bootstrap_metric_intervals(y: np.ndarray, pred: np.ndarray, n_boot: int = 1000) -> pd.DataFrame:
239     rng = np.random.default_rng(SEED)
240     rows = []
241     n = len(y)
242     for _ in range(n_boot):
243         idx = rng.integers(0, n, n)
244         rows.append(regression_metrics(y[idx], pred[idx]))
245     values = pd.DataFrame(rows)
246     return pd.DataFrame(
247         {
248             "metric": values.columns,
249             "estimate": [regression_metrics(y, pred)[c] for c in values.columns],
250             "ci_2.5": values.quantile(0.025).to_numpy(),
251             "ci_97.5": values.quantile(0.975).to_numpy(),
252         }
253     )

```

B.3、 问题二：约束优化与局部精化

Listing 4 q2_optimize.py: 水肥网格评价、预算约束与局部精化

```
89 def evaluate_grid(  
90     model: FittedRegressor,  
91     representative: pd.Series,  
92     water_bounds: tuple[float, float],  
93     fertilizer_bounds: tuple[float, float],  
94     n_water: int,  
95     n_fertilizer: int,  
96 ) -> pd.DataFrame:  
97     water_values = np.linspace(*water_bounds, n_water)  
98     fertilizer_values = np.linspace(*fertilizer_bounds, n_fertilizer)  
99     w, f = np.meshgrid(water_values, fertilizer_values, indexing="xy")  
100     flat_w, flat_f = w.ravel(), f.ravel()  
101     frame = candidate_frame(representative, model, flat_w, flat_f)  
102     pred = model.predict(frame)  
103     return pd.DataFrame({WATER: flat_w, FERTILIZER: flat_f, "predicted_yield": pred})  
104  
105  
106 def choose_budget_optimum(  
107     grid: pd.DataFrame,  
108     water_price: float,  
109     fertilizer_price: float,  
110     budget: float,  
111 ) -> pd.Series:  
112     work = grid.copy()  
113     work["cost_rmb_per_ha"] = economic_cost_per_hectare(  
114         work[WATER], work[FERTILIZER], water_price, fertilizer_price  
115     )  
116     feasible = work[work["cost_rmb_per_ha"] <= budget + 1e-9].copy()  
117     if feasible.empty:  
118         raise RuntimeError("No feasible water/fertilizer combination under the confirmed budget")  
119     feasible = feasible.sort_values(  
120         ["predicted_yield", "cost_rmb_per_ha"], ascending=[False, True]  
121     )  
122     return feasible.iloc[0]  
123  
124  
125 def local_refinement(  
126     model: FittedRegressor,  
127     representative: pd.Series,  
128     start: pd.Series,  
129     water_bounds: tuple[float, float],  
130     fertilizer_bounds: tuple[float, float],  
131     water_price: float,  
132     fertilizer_price: float,  
133     budget: float,  
134 ) -> dict[str, float]:  
135     scale_w = water_bounds[1] - water_bounds[0]  
136     scale_f = fertilizer_bounds[1] - fertilizer_bounds[0]  
137  
138     def objective(z: np.ndarray) -> float:  
139         w = water_bounds[0] + np.clip(z[0], 0, 1) * scale_w  
140         f = fertilizer_bounds[0] + np.clip(z[1], 0, 1) * scale_f  
141         cost = float(economic_cost_per_hectare(w, f, water_price, fertilizer_price))  
142         penalty = 50.0 * max(0.0, (cost - budget) / max(budget, 1.0)) ** 2  
143         frame = candidate_frame(representative, model, np.array([w]), np.array([f]))  
144         return -float(model.predict(frame)[0]) + penalty  
145  
146     z0 = np.array(  
147         [  
148             (float(start[WATER]) - water_bounds[0]) / scale_w,  
149             (float(start[FERTILIZER]) - fertilizer_bounds[0]) / scale_f,
```

```

150     ]
151 )
152 result = minimize(
153     objective,
154     z0,
155     method="Nelder-Mead",
156     options={"maxiter": 500, "xatol": 1e-6, "fatol": 1e-5},
157 )
158 z = np.clip(result.x, 0, 1)
159 w = water_bounds[0] + z[0] * scale_w
160 f = fertilizer_bounds[0] + z[1] * scale_f
161 cost = float(economic_cost_per_hectare(w, f, water_price, fertilizer_price))
162 if cost > budget + 1e-6:
163     return {
164         "water": float(start[WATER]),
165         "fertilizer": float(start[FERTILIZER]),
166         "predicted_yield": float(start["predicted_yield"]),
167         "cost": float(start["cost_rmb_per_ha"]),
168         "success": False,
169         "message": "local search returned infeasible point; dense-grid optimum retained",
170     }
171 frame = candidate_frame(representative, model, np.array([w]), np.array([f]))
172 pred = float(model.predict(frame)[0])
173 if pred + 1e-9 < float(start["predicted_yield"]):
174     return {
175         "water": float(start[WATER]),
176         "fertilizer": float(start[FERTILIZER]),
177         "predicted_yield": float(start["predicted_yield"]),
178         "cost": float(start["cost_rmb_per_ha"]),
179         "success": False,
180         "message": "local search did not improve dense-grid optimum",
181     }
182 return {
183     "water": float(w),
184     "fertilizer": float(f),
185     "predicted_yield": pred,
186     "cost": cost,
187     "success": bool(result.success),
188     "message": str(result.message),
189 }

```

B.4、 问题三：气候情景与稳健决策

Listing 5 q3_robust.py: 情景构造、组合评价与 max-min 选择

```

63 def build_scenarios(train: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
64     features = [c for c in CLIMATE_SCENARIO_FEATURES if c in train.columns]
65     x = train[features].copy()
66     medians = x.median(numeric_only=True)
67     x = x.fillna(medians)
68     scaler = RobustScaler().fit(x)
69     scaled = scaler.transform(x)
70     pca = PCA(n_components=0.90, svd_solver="full", random_state=SEED).fit(scaled)
71     z = pca.transform(scaled)
72     medoid_positions, labels, loss = pam_kmedoids(z, k=3, n_starts=25)
73
74     rows = []
75     for cluster, position in enumerate(medoid_positions):
76         source_index = int(train.index[position])
77         row = {
78             "scenario_id": cluster + 1,
79             "source_index": source_index,
80             "cluster_n": int(np.sum(labels == cluster)),

```

```

81         "probability": float(np.mean(labels == cluster)),
82     }
83     for feature in CLIMATE_FEATURES:
84         if feature in train.columns:
85             row[feature] = train.iloc[position][feature]
86     rows.append(row)
87     scenarios = pd.DataFrame(rows)
88
89     # Name after clustering using observed properties; no preassigned dry/normal/wet labels.
90     rain_rank = scenarios["Rainfall_Seasonal_mm"].rank(method="first").astype(int)
91     temp_rank = scenarios["Temp_Avg_C"].rank(method="first").astype(int)
92     rain_labels = {1: "lower-rain", 2: "middle-rain", 3: "higher-rain"}
93     temp_labels = {1: "cooler", 2: "moderate-temperature", 3: "hotter"}
94     names = []
95     for i in range(len(scenarios)):
96         names.append(
97             f"{rain_labels[rain_rank.iloc[i]]}/{temp_labels[temp_rank.iloc[i]]} representative"
98         )
99     scenarios.insert(1, "scenario_name", names)
100     state = {
101         "features": features,
102         "medians": medians.to_dict(),
103         "scaler": scaler,
104         "pca": pca,
105         "z": z,
106         "labels": labels,
107         "medoid_positions": medoid_positions,
108         "loss": loss,
109         "explained_variance": float(pca.explained_variance_ratio_.sum()),
110     }
111     return scenarios, state
112
113
114 def scenario_candidate_frame(
115     representative: pd.Series,
116     model: FittedRegressor,
117     scenario_row: pd.Series,
118     variety: str,
119     water: np.ndarray,
120     fertilizer: float,
121 ) -> pd.DataFrame:
122     frame = candidate_frame(
123         representative,
124         model,
125         np.asarray(water, dtype=float),
126         np.full(len(water), float(fertilizer)),
127     )
128     for feature in CLIMATE_FEATURES:
129         if feature in frame.columns and feature in scenario_row.index:
130             frame[feature] = scenario_row[feature]
131     frame["Variety"] = variety
132     return frame
133
134
135 def evaluate_variety_water(
136     model: FittedRegressor,
137     representative: pd.Series,
138     scenarios: pd.DataFrame,
139     varieties: list[str],
140     water_bounds: tuple[float, float],
141     fertilizer: float,
142     n_water: int = 501,
143 ) -> pd.DataFrame:
144     water_grid = np.linspace(*water_bounds, n_water)
145     batches = []

```

```

146     for variety in varieties:
147         for _, scenario in scenarios.iterrows():
148             frame = scenario_candidate_frame(
149                 representative, model, scenario, variety, water_grid, fertilizer
150             )
151             frame["__decision_variety"] = variety
152             frame["__scenario_id"] = int(scenario["scenario_id"])
153             frame["__decision_water"] = water_grid
154             batches.append(frame)
155     all_candidates = pd.concat(batches, ignore_index=True)
156     all_candidates["__prediction"] = model.predict(all_candidates)
157     wide = all_candidates.pivot_table(
158         index=["__decision_variety", "__decision_water"],
159         columns="__scenario_id",
160         values="__prediction",
161         aggfunc="first",
162     ).reset_index()
163     probability = scenarios.set_index("scenario_id")["probability"].astype(float)
164     output = pd.DataFrame(
165         {
166             "Variety": wide["__decision_variety"],
167             "WATER": wide["__decision_water"].astype(float),
168         }
169     )
170     scenario_columns = []
171     for scenario_id in sorted(int(x) for x in probability.index):
172         column = f"scenario_{scenario_id}_yield"
173         output[column] = wide[scenario_id].astype(float)
174         scenario_columns.append(column)
175     output["worst_yield"] = output[scenario_columns].min(axis=1)
176     output["expected_yield"] = sum(
177         float(probability.loc[scenario_id]) * output[f"scenario_{scenario_id}_yield"]
178         for scenario_id in sorted(int(x) for x in probability.index)
179     )
180     return output
181
182
183 def select_decisions(evaluation: pd.DataFrame) -> tuple[pd.Series, pd.Series]:
184     robust = evaluation.sort_values(
185         ["worst_yield", "expected_yield"], ascending=[False, False]
186     ).iloc[0]
187     expected = evaluation.sort_values(
188         ["expected_yield", "worst_yield"], ascending=[False, False]
189     ).iloc[0]
190     return robust, expected

```

B.5、 问题四： Pareto 前沿与多目标求解

Listing 6 q4_pareto.py: 环境压力、非支配前沿、折中点与 NSGA-II

```

26 def environmental_pressure(
27     water: np.ndarray | float,
28     fertilizer: np.ndarray | float,
29     water_bounds: tuple[float, float],
30     fertilizer_bounds: tuple[float, float],
31     lambda_water: float = 0.5,
32 ) -> np.ndarray:
33     w = (np.asarray(water, dtype=float) - water_bounds[0]) / (water_bounds[1] - water_bounds[0])
34     f = (np.asarray(fertilizer, dtype=float) - fertilizer_bounds[0]) / (
35         fertilizer_bounds[1] - fertilizer_bounds[0]
36     )
37     w = np.clip(w, 0.0, 1.0)
38     f = np.clip(f, 0.0, 1.0)

```

```

39     return lambda_water * w**2 + (1.0 - lambda_water) * f**2
40
41
42 def nondominated_front(grid: pd.DataFrame) -> pd.DataFrame:
43     ordered = grid.sort_values(["environmental_pressure", "predicted_yield"], ascending=[True, False])
44     keep = []
45     best_yield = -np.inf
46     for idx, row in ordered.iterrows():
47         value = float(row["predicted_yield"])
48         if value > best_yield + 1e-10:
49             keep.append(idx)
50             best_yield = value
51     return ordered.loc[keep].sort_values("environmental_pressure").reset_index(drop=True)
52
53
54 def select_ideal_distance(front: pd.DataFrame) -> tuple[pd.Series, pd.DataFrame]:
55     work = front.copy()
56     y_range = max(work["predicted_yield"].max() - work["predicted_yield"].min(), 1e-12)
57     e_range = max(work["environmental_pressure"].max() - work["environmental_pressure"].min(), 1e-12)
58     work["yield_loss_normalized"] = (
59         work["predicted_yield"].max() - work["predicted_yield"]
60     ) / y_range
61     work["environment_normalized"] = (
62         work["environmental_pressure"] - work["environmental_pressure"].min()
63     ) / e_range
64     work["distance_to_ideal"] = np.sqrt(
65         work["yield_loss_normalized"] ** 2 + work["environment_normalized"] ** 2
66     )
67     return work.loc[work["distance_to_ideal"].idxmin()], work
68
69
70 def weighted_sum_baseline(grid: pd.DataFrame) -> pd.DataFrame:
71     y = grid["predicted_yield"]
72     e = grid["environmental_pressure"]
73     y_norm = (y - y.min()) / max(y.max() - y.min(), 1e-12)
74     e_norm = (e - e.min()) / max(e.max() - e.min(), 1e-12)
75     rows = []
76     for yield_weight in np.linspace(0, 1, 21):
77         utility = yield_weight * y_norm - (1 - yield_weight) * e_norm
78         idx = utility.idxmax()
79         rows.append({"yield_weight": yield_weight, **grid.loc[idx].to_dict()})
80     return pd.DataFrame(rows).drop_duplicates([WATER, FERTILIZER])
81
82
83 class YieldEnvironmentProblem(Problem):
84     def __init__(
85         self,
86         model: FittedRegressor,
87         representative: pd.Series,
88         water_bounds: tuple[float, float],
89         fertilizer_bounds: tuple[float, float],
90         lambda_water: float,
91     ) -> None:
92         super().__init__(
93             n_var=2,
94             n_obj=2,
95             n_ieq_constr=0,
96             xl=np.array([water_bounds[0], fertilizer_bounds[0]]),
97             xu=np.array([water_bounds[1], fertilizer_bounds[1]]),
98         )
99         self.response_model = model
100         self.representative = representative
101         self.water_bounds = water_bounds
102         self.fertilizer_bounds = fertilizer_bounds
103         self.lambda_water = lambda_water

```

```

104
105 def _evaluate(self, x: np.ndarray, out: dict, *args, **kwargs) -> None:
106     frame = candidate_frame(
107         self.representative, self.response_model, x[:, 0], x[:, 1]
108     )
109     y = self.response_model.predict(frame)
110     e = environmental_pressure(
111         x[:, 0],
112         x[:, 1],
113         self.water_bounds,
114         self.fertilizer_bounds,
115         self.lambda_water,
116     )
117     out["F"] = np.column_stack([-y, e])
118
119
120 def nsga2_comparator(
121     model: FittedRegressor,
122     representative: pd.Series,
123     water_bounds: tuple[float, float],
124     fertilizer_bounds: tuple[float, float],
125     lambda_water: float,
126 ) -> pd.DataFrame:
127     problem = YieldEnvironmentProblem(
128         model, representative, water_bounds, fertilizer_bounds, lambda_water
129     )
130     result = minimize(
131         problem,
132         NSGA2(pop_size=120, eliminate_duplicates=True),
133         termination=("n_gen", 140),
134         seed=SEED,
135         verbose=False,
136         save_history=False,
137     )
138     x = np.asarray(result.X)
139     frame = candidate_frame(representative, model, x[:, 0], x[:, 1])
140     return pd.DataFrame(
141         {
142             "WATER": x[:, 0],
143             "FERTILIZER": x[:, 1],
144             "predicted_yield": model.predict(frame),
145             "environmental_pressure": environmental_pressure(
146                 x[:, 0], x[:, 1], water_bounds, fertilizer_bounds, lambda_water
147             ),
148         }
149     ).sort_values("environmental_pressure")

```

B.6、 问题五：混合效应校正与综合排序

Listing 7 q5_adaptation.py: 混合效应模型、跨情景评价与 CRITIC-TOPSIS

```

83 def fit_adjustment_model(work: pd.DataFrame):
84     z_terms = " + ".join(f"z{i}" for i in range(len(MIXED_COVARIATES)))
85     formula = f"{TARGET} ~ C(combo) + {z_terms}"
86     attempts = []
87     for method in ("lbfgs", "powell"):
88         try:
89             with warnings.catch_warnings(record=True) as caught:
90                 warnings.simplefilter("always")
91                 model = smf.mixedlm(
92                     formula,
93                     data=work,
94                     groups=work["year_group"],

```



```

95         re_formula="1",
96         vc_formula={"cluster": "0 + C(cluster_group)"},
97     )
98     result = model.fit(reml=False, method=method, maxiter=1200, disp=False)
99     attempts.append(
100         {
101             "model": "MixedLM",
102             "method": method,
103             "converged": bool(getattr(result, "converged", False)),
104             "warnings": " | ".join(str(x.message) for x in caught),
105         }
106     )
107     if bool(getattr(result, "converged", False)):
108         return result, "MixedLM", attempts
109     except Exception as exc:
110         attempts.append(
111             {"model": "MixedLM", "method": method, "converged": False, "warnings": repr(exc)}
112         )
113
114     # Transparent failure path: fixed year and cluster effects retain adjustment
115     # but no longer estimate partial pooling. The fallback is explicitly logged.
116     formula_fallback = formula + " + C(year_group) + C(cluster_group)"
117     result = smf.ols(formula_fallback, data=work).fit(cov_type="HC3")
118     attempts.append(
119         {"model": "OLS fixed-effect fallback", "method": "HC3", "converged": True, "warnings": ""}
120     )
121     return result, "OLS fixed-effect fallback", attempts
122
123
124 def adjusted_combo_means(result, model_type: str, work: pd.DataFrame) -> pd.DataFrame:
125     combos = sorted(work["combo"].unique())
126     rows = []
127     design_info = result.model.data.design_info
128     fe_names = list(result.fe_params.index) if model_type == "MixedLM" else list(result.params.index)
129     cov = result.cov_params().loc[fe_names, fe_names].to_numpy()
130     reference_year = sorted(work["year_group"].unique())[0]
131     reference_cluster = sorted(work["cluster_group"].unique())[0]
132     for combo in combos:
133         row = {"combo": combo, "year_group": reference_year, "cluster_group": reference_cluster}
134         row.update({"f"z{i}": 0.0 for i in range(len(MIXED_COVARIATES))})
135         new = pd.DataFrame([row])
136         design = np.asarray(build_design_matrices([design_info], new)[0])[0]
137         # For OLS fallback the design includes reference-group fixed effects set to zero.
138         vector = design[: len(fe_names)]
139         params = result.fe_params.to_numpy() if model_type == "MixedLM" else result.params.to_numpy()
140         estimate = float(np.dot(vector, params))
141         se = float(np.sqrt(max(np.dot(vector, cov @ vector), 0.0)))
142         variety, season = combo.split(" | ", 1)
143         rows.append(
144             {
145                 "Variety": variety,
146                 "Season": season,
147                 "combo": combo,
148                 "adjusted_mean": estimate,
149                 "adjusted_se": se,
150                 "adjusted_ci_2.5": estimate - 1.96 * se,
151                 "adjusted_ci_97.5": estimate + 1.96 * se,
152             }
153         )
154     return pd.DataFrame(rows)
155
156
157 def covariate_adjusted_observations(result, model_type: str, work: pd.DataFrame) -> pd.Series:
158     params = result.fe_params if model_type == "MixedLM" else result.params
159     contribution = np.zeros(len(work))

```

```

160     for i in range(len(MIXED_COVARIATES)):
161         contribution += float(params.get(f"z{i}", 0.0)) * work[f"z{i}"].to_numpy()
162     return work[TARGET] - contribution
163
164
165 def scenario_relative_performance(
166     model: FittedRegressor,
167     representative: pd.Series,
168     scenarios: pd.DataFrame,
169     combos: pd.DataFrame,
170     water: float,
171     fertilizer: float,
172 ) -> tuple[pd.DataFrame, pd.DataFrame]:
173     rows = []
174     for _, combo in combos.iterrows():
175         for _, scenario in scenarios.iterrows():
176             frame = scenario_candidate_frame(
177                 representative,
178                 model,
179                 scenario,
180                 str(combo["Variety"]),
181                 np.array([water]),
182                 fertilizer,
183             )
184             frame["Season"] = str(combo["Season"])
185             rows.append(
186                 {
187                     "combo": combo["combo"],
188                     "scenario_id": int(scenario["scenario_id"]),
189                     "predicted_yield": float(model.predict(frame)[0]),
190                 }
191             )
192     detailed = pd.DataFrame(rows)
193     detailed["relative_to_scenario_best"] = detailed.groupby("scenario_id")["predicted_yield"].transform(
194         lambda s: s / s.max()
195     )
196     summary = (
197         detailed.groupby("combo")
198         .agg(
199             worst_scenario_yield=("predicted_yield", "min"),
200             worst_scenario_relative=("relative_to_scenario_best", "min"),
201         )
202         .reset_index()
203     )
204     return summary, detailed
205
206
207 def select_nonredundant_metrics(metric_table: pd.DataFrame) -> tuple[list[str], pd.DataFrame]:
208     candidates = ["adjusted_mean", "adjusted_q10", "stability_score", "worst_scenario_relative"]
209     corr = metric_table[candidates].corr(method="spearman")
210     priority = ["adjusted_mean", "stability_score", "worst_scenario_relative", "adjusted_q10"]
211     selected = []
212     for candidate in priority:
213         if all(abs(float(corr.loc[candidate, kept])) < 0.95 for kept in selected):
214             selected.append(candidate)
215     if len(selected) < 2:
216         selected = ["adjusted_mean", "stability_score"]
217     return selected, corr
218
219
220 def critic_topsis(metric_table: pd.DataFrame, metrics: list[str]) -> tuple[pd.DataFrame, pd.DataFrame]:
221     values = metric_table[metrics].astype(float)
222     normalized = (values - values.min()) / (values.max() - values.min()).replace(0, 1)
223     corr = normalized.corr().fillna(0.0)
224     contrast = normalized.std(ddof=0) * (1.0 - corr).sum(axis=1)

```

```

225     if float(contrast.sum()) <= 1e-12:
226         weights = pd.Series(1 / len(metrics), index=metrics)
227     else:
228         weights = contrast / contrast.sum()
229     weighted = normalized * weights
230     ideal_best = weighted.max(axis=0)
231     ideal_worst = weighted.min(axis=0)
232     d_pos = np.sqrt(((weighted - ideal_best) ** 2).sum(axis=1))
233     d_neg = np.sqrt(((weighted - ideal_worst) ** 2).sum(axis=1))
234     closeness = d_neg / np.clip(d_pos + d_neg, 1e-12, None)
235     ranked = metric_table.copy()
236     ranked["topsis_score"] = closeness
237     ranked["rank"] = ranked["topsis_score"].rank(ascending=False, method="min").astype(int)
238     weight_table = pd.DataFrame({"metric": weights.index, "critic_weight": weights.values})
239     return ranked.sort_values("rank"), weight_table

```